TEKNOFEST

HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

UÇAN ARABA YARIŞMASI

FİNAL TASARIM RAPORU

TAKIM ADI: MAR

TAKIM ÜYELERİ: Ecem Yıldırım, Batuhan Çokmar

BAŞVURU ID: 468748

İçindekiler

1. Hazırlanan senaryolarda kullanan sensörler ve programlama dili ile ilgili bilgiler 4
I. Kod yazarken kullanılan dil ve özellikleri 4
II. Bu dil yerine başka hangi programlama dili tercih edilebilirdi? 4
III. Barometre Sensörü nasıl çalışır ve GNSS ile farkı nedir? 5
IV. İvmeölçer ve Dönüölçer genel hata karakteristikleri 5
2. Senaryo 1'de, hava aracının B noktasına gidişi için tasarlanan algoritma 6
I. Hava Aracının gitmesi gereken yönün hesaplandığı algoritma 6
II. Hava aracının gitmesi istenilen koordinatlarının öğrenildiği veri yapısı 8
III. İnişe geçmesi için gerekli koşul seçimleri 8
IV. İnişte kullanılan irtifa tespit sensörü 8
V. İnişte yere değme hızının ayarlanması 9
3. Senaryo-4 yasak bölgeden kaçış algoritması 9
I. Yasak bölgelerin tespit edildiği veri yapısı 9
II. Yasak bölgelere girmemek için tasarlanan algoritma 9
4. Senaryo-7 trafikte olan arabaları gözeten algoritma 13
I. Diğer hava araçlarının konumlarının bulunması için kullanılan veri yapısı 13
II. Diğer hava araçlarıyla kaza yapmamak için hazırlanan algoritma 13
5. Senaryo-8 en yakın hastaneyi bulmamızı sağlayan algoritma. 15
I. Hastanelerin olduğu veri yapısı 15
II. En yakın hastanenin seçiminin yapıldığı algoritma 15
6. Senaryo-10 GNSS arızasında inişi tamamlayan algoritma 16
I. GNSS arızasının tespit edildiği veri yapısı. 16
II. GNSS arızası durumunda irtifanın hesaplandığı algoritma. 16
7. Senaryo-11 GNSS arızası seyir yapmaya devam eden algoritma 17
I. Arıza sonrası aracın poziyonunu hesaplayan algoritma 17
8. Senaryo 12'de GNSS arıza durumunu tespit eden algoritma 18
I. GNSS karıştırma (jamming) ve yanıltma (spoofing) nedir? Farkları nelerdir? 18
II. GNSS yanıltma (spoofing) durumunun tespit edildiği algoritma. 19
9. Senaryo 13'te inişi tamamlamak için kullanılacak sensör seçimi algoritması. 19
I. Bu senaryo için inişte irtifa kaynağı olarak seçilen sensörler. 19
III. Diğer sensörlerin seçilmeme sebepleri. 20
IV. İrtifa Kaynağı için sensör seçimi yapan algoritma 20
10. Senaryo 15'de manyetometre veri kaybı yaşandığında gidilmesi gereken hedefe yönelimi sağlayan algoritma. 20
I. Manyetometre veri kaybının tespiti 20
II. Hava aracının gitmesi gereken yönün hesabı 21

III. Hava aracının gittiği yönü hesaplayan algoritma 21
11. Senaryo 16'da şarj durumunu gözeterek şarj istasyonlarının tespiti ve hava aracının oraya yönlendirilmesini sağlayan algoritma 22
I. Batarya durumları veri yapısı 22
II. Hangi gerilim eşiğinde şarj olunması gerektiğinin karar mekanizması 22
III. Şarj İstasyonlarının veri yapısı 23
12. Kaynakça 24
EK-1 Senaryolara Ait Videolar 24
EK-2 Kod 25

TEKNOFEST
İSTANBUL HAVACILIK, UZAY VE TEKNOLOJİ FESTİVALİ

1. Hazırlanan senaryolarda kullanan sensörler ve programlama dili ile ilgili bilgiler

I. Kod yazarken kullanılan dil ve özellikleri

Kodun yazımında Python programlama dili kullanılmıştır. Python, çalıştırıldığında kaynak kodları bir yorumlayıcı tarafından işleme alınan yani yorumlanabilen, interaktif, nesne yönelimli programlama dilidir. Üst düzey basit sözdizimine sahiptir ve modülerliği destekler. Neredeyse her türlü platformda çalışabilir. Modüller, olağandışı durumlar, dinamik yazım, yüksek dinamik veri türleri ve sınıfları ile birlikte çalışır. Birçok sistem çağrısı ve kütüphanesine uygun birden fazla arayüze sahiptir. C, C++ ortamında geliştirilebilir ve programlanabilir arayüz ihtiyacı gibi uygulamalarda genişletilmiş dil olarak kullanılabilir (pythontr.com, 2014).

Açıkça tanımlanmış sözdizilimi, basit yapısı ve sade satırları öğrenilmesini kolaylaştırır. Girintili kod yazımı gerektirmesi sebebiyle okunması kolay bir dildir. Kaynak kodunun bakımı kolaydır. Standart kütüphanesi oldukça geniştir ve bu kütüphanenin büyük kısmı UNIX, Windows ve Macintosh sistemlere kolayca taşınabilir, çapraz platform uyumludur. Interaktif mod, GUI programlama, taşınabilirliği ve genişletilebilirliği diğer özellikleri arasındadır. Tüm önemli ticari veritabanları ile bağlantı kurabilme yeteneğine sahiptir. Ölçeklenebilir bir dildir. Python, fonksiyona ve yapılandırılmış programlama yöntemlerini ve nesneye dayalı progranlamayı destekler, script dili olarak kullanılabilir ve büyük uygulamalar oluşturmak amacıyla bayt kod olarak derlenebilir. Oldukça üst düzey dinamik veri türleri sağlamakla birlikte dinamik tür denetimini desteklemektedir. Otomatik çöp toplama özelliğini destekler ve C, C++, COM, ActiveX, CORBA, Java ile entegre edilebilme özelliğine sahiptir (pythontr.com, 2014).

II. Bu dil yerine başka hangi programlama dili tercih edilebilirdi?

Python yerine daha hakim olduğumuz C++ veya C# kullanmayı tercih edebilirdik. İki turing-complete program arasında algoritma kurmak açısından önemli bir değişiklik olmasa da, Python dilini bilmiyor oluşumuz, bazı koşullarda, örneğin Python'un instanced değişkenleri konusunda zorluk yaşamamıza sebep oldu. Strong typing'e alışık biri için veri tipleri arasındaki dönüşümleri kestirememek zorlayıcı oldu. Ancak bunun günümüzde yaygın olarak kullanılan Python'a alışmak için bir fırsat olduğu düşünülebilir.

III. Barometre Sensörü nasıl çalışır ve GNSS ile farkı nedir?

Barometre, hava basıncının irtifa arttıkça düşmesinden faydalanarak çalışan, atmosfer basıncını ölçerek hava aracının deniz seviyesinden ne kadar yüksekte olduğunu belirleyen bir sensördür. Aneroid ve Cıvalı olmak üzere ikiye ayrılır (Mathew, 2017), içinde bir sıvı bulunmamasından ötürü aneroid tipi havacılıkta kullanılmaktadır (ReadPhysics, 2021). GNSS yani küresel uydu seyrüsefer sistemi ise uydulardan hava aracının konumunu 3 boyutlu olarak belirlemesine yarar (EUSPA, 2021). İki sistem de irtifanın belirlenmesinde kullanılabilecek olsa da birbirinden tamamen farklı çalışma prensiplerine sahiptir.

IV. İvmeölçer ve Dönüölçer genel hata karakteristikleri

İvmeölçer sensörlerinde, sabit kayma hatası, sabit kayma kararsızlığı, orantı katsayısı hatası, orantı katsayısı kararsızlığı, eksenel kaçıklık hatası ve rastgele sensör gürültüsü hata kaynaklarıdır. Sensöre herhangi bir ivme uygulanmadığında ölçülen değere sabit kayma hatası denir. Bu hata; sabit terim, sıcaklıkla uyarılan değişimler ve tekrarlanabilirlik gibi bileşenleri içerir. Hatanın sabit terimi ve sıcaklık sonucu ortaya çıkan değişimler uygun statik kalibrasyon testleri ile tespit edilip düzeltilebilir. Sabit kayma kararsızlığı ise ivmeölçerin çalıştırılma sürecindeki değişimden kaynaklı hataları içerir, ivme ölçerin etkinliği boyunca hatanın bu bileşeni değişir. Sensörün çıkışındaki ve girişindeki değişim arasındaki oran, orantı katsayısıdır. Bu değer idealinde 1 olmakla birlikte sensörün ölçtüğü orantı katsayısı arasındaki fark orantı katsayısı hatasını vermektedir. Bileşenleri sabit terim, sıcaklıkla uyarılan değişimler, asimetri, doğrusal olmama hatasıdır ve ilk iki bileşen statik kalibrasyon testleri ile tahmin edilebilir. İvmeölçer orantı katsayısı kararsızlığı, orantı katsayısındaki zamanla değişimi ifade eder ve ancak uzun süreli testlerle belirlenebilir. Eksenel kaçıklık ise dahili ve harici olmak üzere ikiye ayrılır. Harici eksenel kaçıklık hatasını belirlemek kolaydır. Bu hata her eksen etrafında pozitif ve negatif yönde olmak üzere statik kalibrasyon testi gerçekleştirilerek tespit edilir. Son olarak yüksek frekanslı gürültü terimleri (nicemleme gürültüsü, hız rastgele yürüme ve Gauss-Markov gürültüsü) rastgele sensör gürültüsünü oluşturmaktadır (GÜNHAN, 2014).

Dönüölçer sensörlerinde ise ivmeölçer sensörlerine benzer olarak sabit kayma hatası, sabit kayma kararsızlığı, orantı katsayısı hatası, orantı katsayısı kararsızlığı, eksenel kaçıklık hatası, ivmeye bağlı sabit kayma hatası ve rastgele sensör gürültüsü bulunmaktadır. Dönüölçere herhangi bir giriş uygulanmadığı koşulda belirli bir süre ölçülen değerlerin ortalaması alınarak dönüölçer sabit kayma hatası elde edilir. Hata özellikleri ivmeölçerinkine benzerdir. Orantı

katsayısı, açısal hız girişi uygulanan eksen boyunca dönüölçer hassasiyetini oluşturur. Dinamik kalibrasyon testleri ile belirlenir, bu katsayı kararsızlığında ise uzun süre dinamik testler gerçekleştirilir. Dönüölçerler tam dik hizalanmazsa eksenler etrafında dönüye karşı hassasiyet oluşur ve eksenel kaçıklık hatası meydana gelir. Bu hata her eksen etrafında pozitif ve negatif yönde olmak üzere dinamik kalibrasyon testleri ile belirlenir. MEMS dönüölçer yapısının doğrusal ivmeye karşı duyarlılık göstermesi ile birlikte, sensörün çıkışında eksene uygulanan ivme büyüklüğü ile orantılı bir kayma ortaya çıkar. Statik kalibrasyon testi ile dönüölçerin ivmeye bağlılık katsayısı belirlenir ve bu bağlılık ortadan kaldırılır. Son olarak ivmeölçere benzer şekilde yüksek frekanslı gürültü terimleri (nicemleme gürültüsü, açısal rastgele yürüme ve Gauss-Markov gürültüsü) rastgele sensör gürültüsünü oluşturmaktadır (GÜNHAN, 2014).

2. Senaryo 1'de, hava aracının B noktasına gidişi için tasarlanan algoritma

I. Hava Aracının gitmesi gereken yönün hesaplandığı algoritma

Yarışma kapsamında yazılan kod, algoritmasında değişiklik olmadan 20 senaryonun tamamını gerçekleştirecek şekilde tasarlanmıştır ve EK-2'de sunulmuştur. Bu başlık altında, hava Aracının sadece Senaryo 1 için gitmesi gereken yönün hesaplandığı bölümler aşağıda verilmiştir. Diğer yön bulma fonksiyonları, şarj, yol optimizasyonu, acil durum koşulları ve engelden kaçınma kısımları dahil değildir.

Algoritma	Açıklama
def hedefyon(X,Y,tgtX,tgtY): direct=math.atan2(tgtY-Y,tgtX-X) return(direct)	-"hedefyon" fonksiyonu X ve Y olarak yollanan koordinattan tgtX ve tgtY olarak yollanan koordinata olan yönü arctangent2 fonksiyonu kullanarak bulur.
class NAV: enlem=0 boylam=0 nav=NAV() target=NAV()	-NAV isminde, içinde enlem ve boylam değerleri bulunan class oluşturulur. "nav" ve "target" adında iki değişken bu class yapısında oluşturulur.

Cenlem=cezeri.gnss.enlem Cboylam=cezeri.gnss.boylam yon=cezeri.manyetometre.veri	-Aracın enlem ve boylamı "Cenlem" ve "Cboylam" değişkenlerine atanır. Aracın yön değeri, "yon" değişkenine atanır.
hedefindexi=0 .... if hedefindexi>=0 and hedefindexi<len(cezeri.hedefler): primarytarget=cezeri.hedefler[hedefindexi]	-hedefindexi=0 ile ilk hedefin seçilmesi sağlanır. -hedefindexi 0'dan büyük ve dizinin uzunluğundan küçük ise ana hedef değişkeni "primarytarget", cezeri.hedefler dizisindeki mevcut index'e karşılık gelen değere eşitlenir.

if charge!=True:
target.enlem=primarytarget.enlem
target.boylam=primarytarget.boylam
....
if zoneavoid==False:
nav.enlem=target.enlem
nav.boylam=target.boylam
if inis==False:
direction=hedefyon(Cenlem,Cboylam,nav.enlem,nav.boylam)
	-Şarj önceliği yoksa hedef değişkeni "target" değişkenine eşitlenir
	
	-engelden kaçınma durumu yoksa target değişkeni nav değişkenine eşitlenir.
	
	-iniş yapmıyor isek, yönümüz hedefyon fonksiyonuna aracın konumu ve nav konumu gönderilerek bulunur.

cezeri.don(direction-yon) if (math.degrees(yon)<-90 and math.degrees(direction)>90) or (math.degrees(yon)>90 and math.degrees(direction)<-90): cezeri.don(math.radians(180)+yon-direction)	-cezeri.don komutu kullanılarak araca yön verilir. -Aracın yönü -90 dereceden küçük ve istenilen yön 90 dereceden büyük veya yön 90 dereceden büyük istenen yön 90 dereceden küçük ise ters yöne dönmesi sağlanır.
if abs(yon-direction)<0.005 or (abs(math.degrees(direction))>175 and abs(math.degrees(yon))>175): headingset=2	-Eğer istenen yön ile aracın yönü arasındaki fark 0.005'den küçük ise yön tamamen doğru kabul edilir ve dönme durur. Fark 0.08'dan küçük ise yönün yaklaşık doğru

cezeri.don(0.0) if abs(yon-direction)<0.08: headingset=1	olduğu kabul edilerek aynı değişken 1'e eşitlenir. İlk if değişkenindeki 175 derece ile başlayan koşullar, aracın hem hedefinin hem yönünün -175'den küçük ve +175'den büyük olduğu alanda yon-direction kısmında oluşan büyük fark ile ani dönüşler yaparak salınım yapmasını engellemek için konmuştur.

II. Hava aracının gitmesi istenilen koordinatlarının öğrenildiği veri yapısı

Aracın gitmesini istediğimiz hedefler, cezeri.hedefler dizisi içerisindedir. Bu dizi içerisindeki ilk hedefin enlemi cezeri.hedefler[0].enlem boylamı ise cezeri.hedefler[0].boylam olarak okunabilir. 0 yerine farklı rakamların gelmesi diğer senaryolarda birden fazla hedef verildiğinde bunlara erişmek için kullanılmaktadır.

III. İnişe geçmesi için gerekli koşul seçimleri

Aracın inişe başlaması için gerekli koşullar,

1. vardığımız koordinatta inmek istemediğimiz koşullar (örneğin, bir engelden kaçmak için sadece o koordinata ulaşıp farklı bir yöne uçmak isteyebiliriz),
2. aracın inmek üzere olduğu koordinatın iniş bölgesi olması,
3. hedefimize ilk olarak 10 metreden az mesafeyle yaklaşmamız, iniş boyunca 20 metreden az mesafede kalmamız.

En son inişin gerçekleşmesi için ek koşullar ise,

1. Senaryoda sürenin önemli olup olmamasına göre iniş hedefine 2 veya 8 metre mesafeden az mesafede olmamız,
2. Dikey hızın -3m/s'den daha hızlı olmaması

IV. İnişte kullanılan irtifa tespit sensörü

İniş sırasında yerden yükseklik değerini döndüren gnd() fonksiyonu, arıza durumlarına göre önce radar, sonra lidar, bu iki sensörün bozulması koşulunda da irt() fonksiyonunun döndürdüğü irtifa ile harita bilgisinden okunan "yükselti" değerinin farkını alarak yerden yüksekliği bulmaktadır. irt() fonksiyonu ise arıza durumuna göre barometre, gnss veya imu verisini kullanmaktadır, iniş sırasında bu 5 sensörden sadece birinin çalışması kritik olan yerden yükseklik verisinin bulunması için yeterlidir.

V. İnişte yere değme hızının ayarlanması

İnişte yere yaklaşma üç kademe olarak belirlenmiştir. 15 metreye kadar hızlı yaklaşma, 6 metreye kadar yavaş yaklaşma, 6 metrenin altında ise -1,2m/s iniş hızına denk gelen cezeri.asagi_git(0.3) komutu kullanılmıştır. Sürenin önemli olduğu senaryolarda bu eşik değerleri ve hızlar değişmektedir.

3. Senaryo-4 yasak bölgeden kaçış algoritması

I. Yasak bölgelerin tespit edildiği veri yapısı

Yasak bölgelerin bulunduğu veri yapısı, istenen enlem ve boylam değerleri için cezeri.harita.bolge(enlem,boylam).ucusa_yasakli_bolge şeklindedir. Yerel harita değerleri ise cezeri.yerel_harita[index].ucusa_yasakli_bolge şeklindedir.

II. Yasak bölgelere girmemek için tasarlanan algoritma

"noflytest" fonksiyonu, istenen koordinatlar currX ve currY'den başlayarak, hedefin koordinatları tgtX ve tgtY yönüne doğru, taranması istenen dst mesafesi boyunca yasaklı bölge olup olmadığını kontrol eder.

Algoritma	Açıklama
def noflytest(currX,currY,tgtX,tgtY,dst): nfly=99999 D=hedefyon(currX,currY,tgtX,tgtY) index=0	-fonksiyon tanımı nfly lokal değişkenine çok yüksek bir değer ata, hedefyon ile 1. Noktadan 2. Noktaya olan yönü bul ve D lokal değişkenine eşitle, indexi 0'a eşitle,

while index<dst-2:
nflyen=currX+index*math.cos(D)/20
nflyboy=currY+index*math.sin(D)/20
if cezeri.harita.bolge(nflyen,nflyboy).ucusa_yasakli_bolge==True and nfly>index:
nfly=index
break
if dst>100:
index=index+3
elif dst>40:
index=index+2
	-index dst-2'den düşük olduğu sürece,
	-ilk noktadan, hedef noktaya index uzaklığı mesafedeki enlem ve boylamı hesapla.
	-bu enlem boylamın haritadan yasaklı bölge olup olmadığını kontrol et
	-eğer bu bölge yasaklı bölge ise en yakın yasaklı bölge bulundu, döngüyü sonlandır.
	-Taranan mesafe çok uzak ise yüksek hassasiyete ihtiyacımız yok. Kaynak kullanımını azaltmak için mesafeye göre 3'er

else: index=index+1 return(nfly)	2'şer veya 1'er metre hassasiyet yeterli, indexi buna göre arttır. -nfly lokal değişkenini döndür.

Algoritmanın ana döngü içerisindeki kısmı aşağıdaki gibidir:

Algoritma	Açıklama
if zoneavoid==False: nav.enlem=target.enlem nav.boylam=target.boylam nofly=noflytest(Cenlem,Cboylam,nav.enlem,nav.boylam,distance)	-zoneavoid değişkeni mevcutta kaçınma manevrası yapıp yapmadığımızı kontrol eder. -nav değişkeni, an itibari ile yönlenilen koordinatları tutar. -nofly değişkeni noflytest fonksiyonu ile uçuş yapılmaması gereken bölgeye olan mesafeyi ölçer.

else:
if charge!=True:
nofly=noflytest(Cenlem,Cboylam,primarytarget.enlem,primarytarget.boylam,Dist(Cenlem,Cboylam,primarytarget.enlem,primarytarget.boylam))
else:
nofly=noflytest(Cenlem,Cboylam,target.enlem,target.boylam,Dist(Cenlem,Cboylam,target.enlem,target.boylam))
	-Eğer zoneavoid true ise, nav değişkeni engelden kaçınma koordinatlarını tutar ve araç bu noktaya yönelmiştir. Araç bu koordinata yöneldiği sırada asıl hedefine yolun açılıp açılmadığını kontrol eder. Bu koşulda, şarj talebi varsa target, hedeflenen şarj istasyonu olacağından bu noktaya, eğer şarj talebi yoksa primarytarget değişkeni ana hedefin koordinatlarını tuttuğundan bu noktaya doğru uçuşa yasak bölge kontrolü yapılır.

if nofly>=400: ..... zoneavoid=False nav.enlem=target.enlem nav.boylam=target.boylam	-Bu kontrol sırasında engele 400* metre veya daha büyük mesafe var ise; -kaçış manevrası bittiği için zoneavoid false -nav değişkeni target değişkenine eşitlenir. *burada noflytest fonksiyonunun eğer bir engel bulamazsa 99999 değerini döndürdüğü unutulmamalıdır.
if nofly<300 and zoneavoid==False and nofly<distance: if nofly!=0:	-eğer nofly değişkeni 300'den küçük ve hedeften daha yakın ve an itibarıyla kaçış manevrası yapılmıyor ise:

mu=PTdistance/nofly/3 if mu<1: mu=1 zoneavoid=True angular=0 mult=1	Ana hedefe mesafe ile uçuşa yasak bölgeye olan mesafenin oranına göre "mu" adında bir değişken oluştur. -kaçış manevrası yapılacak -angular ve mult adında iki değişken oluştur.

if waypoints==0:
while angular<=1.7:
testnewdirection=(direction*2+yon)/3+angular*mult
if mult==1:
mult=-1
else:
mult=1
angular=angular+0.15
if prevnofly>nofly:
waypoint1enlem=Cenlem+(prevnofly+10)*math.cos(testnewdirection)/20
waypoint1boylam=Cboylam+(prevnofly+10)*math.sin(testnewdirection)/20
towaypoint1=noflytest(Cenlem,Cboylam,waypoint1enlem,waypoint1boylam,prevnofly+10)
else:
waypoint1enlem=Cenlem+(nofly*mu+20)*math.cos(testnewdirection)/20
waypoint1boylam=Cboylam+(nofly*mu+20)*math.sin(testnewdirection)/20
towaypoint1=noflytest(Cenlem,Cboylam,waypoint1enlem,waypoint1boylam,nofly*mu+20)
prevnofly=nofly
	-Kaçış için bir yol noktası yok ise,
	-Açısal fark 1.7 radyandan küçük ise;
	-2/3 hedef yönü, 1/3 mevcut yön ağırlıklı ortalamasını baz alarak yeni yön oluştur.
	-mult 1 ise -1 yap,
	
	-Mult -1 ise 1 yap ve angular değişkenini 0.15 radyan (yaklaşık 8.6 derece) arttır.
	
	-Önceki nofly mesafesi mevcut nofly mesafesinden fazla ise, uzaklaşıyoruz demektir, eski mesafeye göre bir kontrol yap.
	-Waypoint1, kaçış için olası yeni hedef adayının koordinatlarını tutar.
	
	Değilse, yasak bölgeye yaklaşıyoruz demek, bu durumda kontrolü nofly değişkenine göre yap. Yasak bölgeye mesafe çok uzaksa yaklaşmadan daha az manevra ile kaçılabilir, bu yüzden mu çarpanı ile kontrol edilen mesafeyi arttır.
	"towaypoint1", noflytest fonksiyonu ile elde edilen waypoint1'e yasak bölge olup olmadığını kontrol et.

if towaypoint1>=99999: nav.enlem=waypoint1enlem nav.boylam=waypoint1boylam break waypoints=1	-hesaplama bittiği için prevnofly değişkenini nofly değişkenine eşitle. -Eğer towaypoint 99999 ise istenen koordinata kadar yasak bölge bulunmamaktadır, kaçış manevrası için gerekli koordinat bulundu. -waypoint1 enlem ve boylamını nav değişkenine ata. -döngüyü sonlandır. -kaçış yol noktasını bire eşitle.

Algoritma açıklaması: Algoritma her döngüde örneğin 200 metre uzaktaki 0 derece yöndeki hedefe giderken 198 metreye kadar sürekli noflytest fonksiyonunu çağırarak engel olup olmadığını kontrol ediyor. Örneğin 100 metre mesafede bulmuş olsun. Bu durumda hedefine varamayacağını ve kaçınması gerektiğini anlıyor. Kod yön bulma döngüsüne giriyor, hedef yönünün 2/3'ü mevcut yönün 1/3'ü olarak alınan yönün +0.15 radyan, yani 8,6 derece sağına yönelip 120 metre gidersem yolda engel var mı diye kontrol ediyor. Engel var ise, mult değişkenini değiştirerek, -0.15 yani 8,6 derece solda 120 metre engel olup olmadığını kontrol ediyor. Yine engel varsa, angular değerini 0.15 arttırıyor, bu sefer aynı kontrolü 0.30 radyan yani 17,2 derece yön değişimleri için yapıyor. Waypoints değişkeninin kullanılması, yön bulma kısmını bir fonksiyon içine atarak, birden fazla dönüş noktası belirlemek için düşünülmüş ancak kodda kullanılmamıştır. Senaryo 4 için kodun çalıştırılarak gösterildiği videosu EK-1'de paylaşılmıştır.

Son olarak, yerel harita kullanılarak, asıl yön bulma algoritmasının çok dar alanlarda yandan yaklaşmalara karşı zayıf kalacağı koşullara yönelik bir önlem alınmıştır.

Algoritma	**Açıklama**
while index<len(cezeri.yerel_harita): if cezeri.yerel_harita[index].ucusa_yasakli_bolge ==True ..... : checknofly=True index=index+1	-yerel harita dizisinin boyutu boyunca -eğer yerel haritada uçuşa yasaklı bölge varsa, checknofly değişkeni True
if checknofly==True:	-checknofly true ise:

Csag=collisionavoid(yon+math.radians(-90),10,True,True,False,True)
Csol=collisionavoid(yon+math.radians(90),10,True,True,False,True)
if Csag<Csol and Csag<10:
cezeri.sola_git(cezeri.yavas)
if Csol<Csag and Csag<10:
cezeri.saga_git(cezeri.yavas)
	-Aracın 90 derece sağında ve 90 derece solunda 10 metre mesafede bir engel var mı?
	-Var ise, hangi yönde mesafe daha fazla ise o yöne doğru git, collisionavoid engel yok ise 99999 döndürecektir.

Burada geçen collisionavoid fonksiyonu, yukarıda bahsedilen noflytest fonksiyonu ile birebir aynı mantıkta çalışmaktadır, tek farkı ek olarak trafik ve rüzgar gibi engelleri isteğe bağlı kontrole dahil etmektedir. Bu fonksiyonun kodu, ek özelliklerinin senaryo 4'de kullanılmaması nedeniyle bu bölümde verilmeyecektir, EK-2'de incelenebilir.

4. Senaryo-7 trafikte olan arabaları gözeten algoritma

I. Diğer hava araçlarının konumlarının bulunması için kullanılan veri yapısı

Bir bölgede diğer hava araçlarının bulunup bulunmadığı, bölgenin enlem ve boylamını kullanarak cezeri.harita.bolge(enlem,boylam).trafik değerinin True olup olmadığının sorgusu ile anlaşılmaktadır.

II. Diğer hava araçlarıyla kaza yapmamak için hazırlanan algoritma

Algoritma diğer araçların sağında veya solunda olmasını kontrol ederek aksi yöne ilerletecek, ileri ve geri komutları ile bu araçlara fazla yaklaşmasına engel olacaktır. Bu bölümde açıklanan kod çalışırken aracın yön bulma kodu her zaman aracın yönünü istenen hedefe yönlendirmeye devam edecektir. Senaryo 7 için kodun çalıştırılarak gösterildiği video EK-1'de paylaşılmıştır.

Algoritma	Açıklama

def collisionavoid(D,dst,nf,rz,tr,trb):
nfly=99999
index=0
while index<dst-2:
nflyen=Cenlem+index*math.cos(D)/20
nflyboy=Cboylam+index*math.sin(D)/20
	-Collisionavoid, D yönünde, dst mesafeye kadar, uçuşa elverişli olmayan bütün koşulları isteğe bağlı kontrol eder.
	-nfly çok büyük değer ata, index 0'dan başlat.

.... if tr==True and cezeri.harita.bolge(nflyen,nflyboy).trafik==True and nfly>index: nfly=index break .... index=index+3 return(nfly)	-index dst-2'den küçük ise kontrol edilecek koordinatları bul. -Senaryo-7 için önemli kısım, eğer tr değeri True ise, trafik kontrolü istiyoruz demektir. -bölge trafik bölgesi ise en yakın trafik bölgesi bulundu. Döngüyü sonlandır. -indexi 3 arttır, 3 metre hassasiyet yeterli görülmüştür. -bulunan minimum mesafeyi döndür.

deg=-180
Ctemp=99999
Closestobject=99999
Closestangle=0
while deg<180:
Ctemp=collisionavoid(yon+math.radians(deg),60,False,False,True,False)
if Ctemp<Closestobject:
Closestobject=Ctemp
Closestangle=deg
deg=deg+15
 	
if Closestobject>0 and Closestobject<99999:
chiz=3/(float(Closestobject)/15)
if chiz>cezeri.hizli:
chiz=cezeri.hizli
if Closestobject<30:
if (Closestangle>=-45 and Closestangle<=45):
cezeri.ileri_git(0)
cezeri.geri_git(chiz)
if (Closestangle<-135 and Closestangle>-180) or (Closestangle>135 and Closestangle<180)
cezeri.geri_git(0)
	-deg değişkeni -180 dereceden başlat.
	-Ctemp, geçici değişken ata
	-Closestobject, en yakın objeye mesafe
	-Closestangle, en yakın objeye olan açı
	-Deg değişkeni 180'den küçük ise;
	-60 metreye kadar diğer arabaları bul ve mesafeyi Ctemp'e eşitle.
	-Ctemp En yakın objeden daha yakın ise;
	-yeni en yakın obje Ctemp
	-objenin açısını mevcut deg'e eşitle
	-deg'i 15 arttırarak 15 derece aralıklarla bu kontrolü yap.
	-En yakın obje var ise;
	mesafeye oranlı hız hesabı.
	-Hız değerinin cezeri.hizli değerini geçmesi şartnameye uygun değil.
	-Eğer en yakın obje 30m'den yakınsa
	-Objenin açısı +/-45 derecede ise ileridedir
	-İleri gidişi durdur
	-chiz hızında geri git.
	-Objenin açısı -135'den küçük ve +135'den büyük ise geridedir,
	-Geri gidişi durdur

cezeri.ileri_git(chiz)
if Closestobject<60 and distance>20:
 	
if Closestangle<0: #sag
cezeri.sola_git(chiz)
if Closestangle>=0: #sol
cezeri.saga_git(chiz)
else:
cezeri.sola_git(0)
cezeri.saga_git(0)
else:
cezeri.sola_git(0)
cezeri.saga_git(0)
	-chiz hızında ileri git
	-Eğer en yakın mesafe 60'dan küçük ve hedefe mesafe 20 metreden büyük ise;
	-Eğer obje aracın sağında ise;
	-chiz hızında sola git;
	-Eğer obje aracın sağında veya önünde ise;
	-chiz hızında sağa git;
	Eğer obje 60m'den uzak veya hedefe mesafe 20'den düşük ise yana gitme.
	
	Eğer en yakın obje yok ise yana gitme. ileri hızı ile ilgili müdahale yoktur, bu hız kodun geri kalanı tarafından atanacaktır.

5. Senaryo-8 en yakın hastaneyi bulmamızı sağlayan algoritma.

I. Hastanelerin olduğu veri yapısı

Hastanelerin olduğu veri yapısı bütün hastanelerin koordinatlarının tutulduğu cezeri.harita.hastaneler[] dizisidir. Örneğin, dizideki 3. Hastanenin enlem değeri, cezeri.harita.hastaneler[3].enlem şeklinde okunabilmektedir.

II. En yakın hastanenin seçiminin yapıldığı algoritma

Algoritma	Açıklama
def Dist(X,Y,tgtX,tgtY): dst=math.sqrt((Y-tgtY)**2+(X-tgtX)**2) dst=dst*20 return(dst)	-Dist fonksiyonu iki nokta arasındaki mesafeyi bulur. -Pisagor teoremi ile mesafe bul -Bulunan mesafeyi 20 ile çarparak metreye çevir, döndür.

def getclosesthospital():
EskiMes=999999
j=0
for i in range(len(cezeri.harita.hastaneler)):
	-En yakın hastane bulan fonksiyon.
	-EskiMes çok büyük bir değişken ata.
	-j en yakın hastanenin indeksi.
	-Cezeri.harita.hastaneler listesi boyunca;

Mesafe=Dist(Cenlem,Cboylam,cezeri.harita.hastaneler[i].enlem,cezeri.harita.hastaneler[i].boylam) if Mesafe<EskiMes: EskiMes=Mesafe j=i return(j)	-Dist fonksiyonu kullanarak aracın mevcut konumundan indeksteki hastaneye mesafeyi bul -Eğer mesafe eski mesafeden küçük ise; -Eski mesafeyi mesafeye eşitle; -En yakın hastane artık mevcut indeksteki hastane. -döngü bitince en yakın hastaneyi döndür.
if cezeri.acil_durum==True: yenihedef=False inmekistiyoruz=True primarytarget=cezeri.harita.hastaneler[getclosesthospital()]	-Eğer acil durum var ise; -yeni hedef istemiyoruz. -hedefe varınca inmek istiyoruz -ana hedef hastaneler dizisinden en yakın hastane indeksi ile çağırılan hastane.

6. Senaryo-10 GNSS arızasında inişi tamamlayan algoritma

I. GNSS arızasının tespit edildiği veri yapısı.

GNSS arızası cezeri.gnss.hata değerinin True olması ile anlaşılmaktadır.

II. GNSS arızası durumunda irtifanın hesaplandığı algoritma.

Yazılan algoritmada, irtifa irt() fonksiyonu ile bulunmaktadır ve GNSS ikinci önceliktedir. Fonksiyon dışında, ana döngü içinde IMU değerlerinden hesaplanan INSRint değişkeni global değişkendir. Ana döngü içinde hesaplanmasının sebebi, değer hesaplandıktan hemen sonra eski zaman değeri yazılarak doğruluğunun arttırılmasıdır. Bu şekilde IMU'dan bulunan irtifa ve konum değeri 8 saatlik senaryoda bile doğruluğunu koruması sağlanmıştır.

Algoritma	Açıklama

def irt():
if cezeri.barometre.hata==False
ir=(cezeri.barometre.yukseklik)
elif cezeri.gnss.hata==False and cezeri.gnss.karisim==False:
ir=(cezeri.gnss.irtifa)
else:
ir=INSRirt
return(ir)
	-irtifa bulan fonksiyon
	-eğer barometre çalışıyor ise
	Barometre irtifa değerini oku, ir'ye yaz
	-eğer barometre bozuk, GNSS sorunsuz çalışıyor ise;
	GNSS irtifa değerini oku, ir'ye eşitle
	Eğer hem barometre hem GNSS çalışmıyor ise; IMU değerini kullan ve ir'ye eşitle
	ir'yi döndür

INSRirt=0 timeR=0 .... INSRirt=cezeri.imu.Vz*((float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000000)-timeR)/22+INSRirt .... timeR=float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000000	-Global değişkenler ve başlangıç değerleri -IMU'nun dikey hız değeri kullanılarak irtifa değerinin hesaplanması -Hesaplamadan hemen sonra o anın zamanını oku ve sonraki döngü için sakla.

Açıklama: rospy.get_rostime kullanımı, kodu okunması zor hale getirmekte olduğundan ek açıklamaya ihtiyaç duyulmuştur. Kullanılan formül,
INS_Rospy_irtifa = imu.Vz(m/s) * (yeni_zaman – eski_zaman)/1.1 + INS_Rospy_irtifa
olarak açıklanabilir. Kodun bu kısmı proje dökümantasyonunda henüz cezeri.ros_zamani() fonksiyonu verilmeden önce yazıldığı için doğrudan rostime ile simülasyon zamanı saniye ve nanosaniye olarak okunup birleştirilmiştir.
Yapılan testlerde formül doğru olmasına rağmen IMU kullanılarak bulunan enlem, boylam ve irtifa değerleri, tutarlı olarak %10 hata yaptığı gözlemlenmiştir. Bu yüzden değer 1.1'e bölünerek kullanılmıştır. Bu şekilde yaklaşık 8 saatlik süren 18. senaryoda bile hiç GNSS kullanmadan uçuş tamamlanabilmektedir. Bu durum bir kalibrasyon hatası olarak yorumlanıp böyle bırakılmıştır.

7. Senaryo-11 GNSS arızası seyir yapmaya devam eden algoritma

I. Arıza sonrası aracın poziyonunu hesaplayan algoritma

Algoritma	Açıklama
INSRenlem=cezeri.gnss.enlem INSRboylam=cezeri.gnss.boylam timeR=0 .... INSRenlem=cezeri.imu.Vx*((float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000000)-timeR)/22+INSRenlem INSRboylam=cezeri.imu.Vy*((float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000000)-timeR)/22+INSRboylam	Enlem ve boylam için global değişkenleri tanımla ve başlangıç değerlerini GNSS'den oku. -IMU Vx yatay hız değeri, bir önceki döngüden gelen zaman ve anlık okunan zaman kullanılarak enlemi hesapla. -IMU Vy yatay hız değeri kullanılarak boylamı hesapla.

timeR=float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000000	timeR, rospy simülasyon zamanını hesaptan hemen sonra oku ve bir sonraki döngü için kaydet.

if cezeri.gnss.hata==False and cezeri.gnss.karisim==False and abs(cezeri.gnss.enlem-INSRenlem)<0.2:
Cenlem=cezeri.gnss.enlem
INSRenlem=cezeri.gnss.enlem
else:
Cenlem=INSRenlem
if cezeri.gnss.hata==False and cezeri.gnss.karisim==False and abs(cezeri.gnss.enlem-INSRboylam)<0.2:
Cboylam=cezeri.gnss.boylam
INSRboylam=cezeri.gnss.boylam
else:
Cboylam=INSRboylam
	-eğer gnss enlem verisini okumakta herhangi bir problem yok ise;
	-Aracın enlemini GNSS verisine eşitle
	-Hesaplanan IMU enlem verisini yoksay ve GNSS enlem verisine eşitle
	-Problem var ise;
	-Aracın enlemini IMU enlem verisine eşitle.
	-eğer gnss boylam verisini okumakta herhangi bir problem yok ise;
	-Aracın boylamını GNSS verisine eşitle
	-Hesaplanan IMU boylam verisini yoksay ve GNSS boylam verisine eşitle
	-Problem var ise;
	Aracın enlemini IMU boylam verisine eşitle

Açıklama: rospy.get_rostime kullanımı kodu okunması zor hale getirmekte olduğundan ek açıklamaya ihtiyaç duyulmuştur. Kullanılan formül,
INS_Rospy_Enlem = imu.Vx(m/s) * (yeni_zaman – eski_zaman)/22 + INS_Rospy_Enlem
olarak açıklanabilir. Burada bir önceki bölümde bahsedilen %10'luk hata devam etmektedir. Bu yüzden önce 20m olan sektör boyutuna, sonra 1.1'e bölmek yerine doğurdan 22'ye bölünmesi seçilmiştir.

8. Senaryo 12'de GNSS arıza durumunu tespit eden algoritma

I. GNSS karıştırma (jamming) ve yanıltma (spoofing) nedir? Farkları nelerdir?

GNSS karıştırma, bir dış etken sonucu GNSS sinyalinin araca ulaşamaması anlamına gelmektedir. GNSS yanıltma ise, GNSS sinyalinin araca yanlış veri sağlayacak şekilde gönderilmesi olarak açıklanabilir. Algoritma açısından farkı, GNSS ekipmanı karıştırma yapıldığında algılayabilir, ancak yanıltma yapıldığında GNSS sorunsuz çalışıyor görünüp yanlış konum verisi yollayabilmektedir.

II. GNSS yanıltma (spoofing) durumunun tespit edildiği algoritma.

Algoritma 7. Bölümdeki algoritma ile aynıdır, önemli bölümler işaretlenmiştir. Senaryo 12 için kodun çalıştırılarak gösterildiği video EK-1'de paylaşılmıştır.

Algoritma	Açıklama
if cezeri.gnss.hata==False and cezeri.gnss.karisim==False and **abs(cezeri.gnss.enlem-** **INSRenlem)<0.2:** Cenlem=cezeri.gnss.enlem INSRenlem=cezeri.gnss.enlem **else:** **Cenlem=INSRenlem** if cezeri.gnss.hata==False and cezer i.gnss.karisim==False and **abs(cezeri.gnss.enlem-** **INSRboylam)<0.2:** Cboylam=cezeri.gnss.boylam INSRboylam=cezeri.gnss.boylam **else:** **Cboylam=INSRboylam**	-eğer GNSS sisteminde hata yok ise ve, -eğer GNSS karışım yok ise; -eğer GNSS'den okunan enlem/boylam verisi ile IMU ile hesaplanan veri arasındaki fark 0.2'den küçük ise GNSS'in sağladığı veri doğru kabul edilir. Kodun ana döngüsü 100ms bekleme yaparak çalışmaktadır, 0.2'lik bir fark 20m sektör boyu ile çarpıldığında yaklaşık 40m/s hıza denk gelmektedir. Hava aracı bu hıza çıkamamaktadır. Eğer GNSS verisi ile IMU dan hesaplanan değer arasında 0.2'den büyük bir fark oluşmuş ise algoritma, GNSS'in yanıltmaya maruz kaldığını ve güvenilmemesi gerektiğini kabul eder ve IMU değerini kullanır. Bu kalıcı bir durum değildir, hatanın varlığı her döngüde kontrol edilir ve algoritma GNSS'e yapılan yanıltma sonlandığında GNSS verisi yeniden kullanılmaya başlanacaktır.

9. Senaryo 13'te inişi tamamlamak için kullanılacak sensör seçimi algoritması.

I. Bu senaryo için inişte irtifa kaynağı olarak seçilen sensörler.

Algoritma, temel tasarımı gereği senaryo bazlı bir sensör seçimi yapmamaktadır. irt() fonksiyonu aracın üstünde buluntan irtifayı arızalı olup olmamasına göre sırasıyla barometre, GNSS ve IMU'dan, gnd() fonksiyonu ise (içinde irt() fonksiyonunu da çağırdığı için) yerden yüksekliği sırasıyla radar, lidar, barometre, GNSS veya IMU'dan ölçmekte veya hesaplamaktadır. Sensörlerde oluşan arıza durumları hava aracının uçuşunu (iniş kalkış, yön bulma) belirleyen algoritmaya yansıtılmadan sonucu gönderilmektedir.

Simülasyon ortamında yapılan testlerde radar barometre ve GNSS bozulduğu durumda, irtifa

için IMU sensörünün, yerden yükseklik için genel olarak de Lidar'ın kullanıldığı gözlemlenmiştir. Bazı istisnai durumlarda iniş bölgesinde yağmur bulunması Lidar da çalışmadığından yerden yükseklik de IMU aracılığı ile bulunmuştur.

III. Diğer sensörlerin seçilmeme sebepleri.

Algoritmada bir sensörün vermesi gereken veri, eğer bir başka sensör tarafından veriliyorsa veya matematiksel olarak hesaplanabiliyorsa olası arıza koşullarına karşı alternatif olarak değerlendirilmiştir. Bu kullanım bir sensörün seçilmemesinin tek sebebi o sensörün istenen veriyi hesaplamakta kullanılamayacak olmasıdır. Örneğin manyetometre sensörünü irtifa hesabına dahil etmek mümkün değildir.

IV. İrtifa Kaynağı için sensör seçimi yapan algoritma

Barometrik irtifa döndüren irt() fonksiyonunun çalışması, Senaryo 10 ile ilgili 6. Bölüm'de anlatılmıştır, aynı algoritma bu senaryo için de geçerlidir. Yerden irtifa ölçen gnd() fonksiyonunun algoritmasi ise aşağıdaki gibidir.

Algoritma	Açıklama

def gnd():
  if cezeri.radar.hata==False:
  gn=cezeri.radar.mesafe
  elif cezeri.lidar.hata==False:
  gn=cezeri.lidar.mesafe
  else:
  gn=irt()-(cezeri.harita.bolge(Cenlem,Cboylam).yukselti)
 	
  return(gn)
	-gnd() fonksiyonu yerden yüksekliği bulmaktadır
	-Radar çalışıyor ise;
	-Radar ile ölçülen mesafeyi kullan, gn'ye eşitle
	-Radar bozuk lidar çalışıyor ise;
	-Lidar ile ölçülen mesafeyi kullan, gn'ye eşitle
	-Hem radar hem Lidar bozuk ise;
	-irt() fonksiyonu ile irtifa'dan aracın bulunduğu bölgedeki yükseltiyi çıkar. GNSS bozuk ise aracın enlem ve boylamı da 7. Bölümde anlatıldığı şekilde IMU ile bulunacaktır ve gn'ye eşitle
	-gn'yi döndür.

10. Senaryo 15'de manyetometre veri kaybı yaşandığında gidilmesi gereken hedefe yönelimi sağlayan algoritma.

I. Manyetometre veri kaybının tespiti

Manyotemetredeki veri kaybı, cezeri.manyetometre.hata değişkeninin True olması ile tespit edilir.

II. Hava aracının gitmesi gereken yönün hesabı

Algoritma temel tasarımı gereği sensör arızalarını yön hesaplayan algoritmaya yansıtmamaktadır. Bu algoritma Senaryo 1'in anlatıldığı 2. Bölüm ile aynıdır:

Algoritma	Açıklama
direction=hedefyon(Cenlem,Cboylam,nav.enlem,nav.boylam)	Senaryo 2'de anlatılan "hedefyon" fonksiyonuna aracın enlem boylamı ve gidilmek istenen koordinatın enlem boylamı kullanılarak aracın gitmesi istenen yön hesaplanır.
cezeri.don(direction-yon) if (math.degrees(yon)<-90 and math.degrees(direction)>90) or (math.degrees(yon)>90 and math.degrees(direction)<-90): cezeri.don(math.radians(180)+yon-direction)	-cezeri.don komutu kullanılarak araca yön verilir. Aracın yönü -90 dereceden küçük ve istenilen yön 90 dereceden büyük ise ters yöne dönmesi sağlanır.

III. Hava aracının gittiği yönü hesaplayan algoritma

Algoritma	Açıklama

INSRang=INSRang+cezeri.imu.acisal_hiz*((float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000000)-timeR)*1.01
timeR=float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000000
....
if cezeri.manyetometre.hata==False and cezeri.manyetometre.gurultu==False:
yon=cezeri.manyetometre.veri
INSRang=cezeri.manyetometre.veri
else:
if (abs(cezeri.imu.Vx)>4 or abs(cezeri.imu.Vy)>4) and headingset>0 and inis==False and Closestobject>60:
	-IMU açısal hız verisini ve simülasyon zamanını kullanarak aracın yönünü bul.
	-1.01 kalibrasyon için çarpım.
	-Rospy simülasyon zamanını bir sonraki döngü için bul ve kaydet.
	
	-Manyetometre hatası yok ise ve manyetometre verisi güvenilir ise:
	-yon ve IMU ile bulunan INSRang değişkenlerini manyetometre verisine eşitle
	-manyetometre kullanılamaz ise;
	Eğer aracın hızı yatayda veya dikeyde 4m/s üstünde ise, dönmüyorsa, inmiyorsa ve başka objelerden kaçmıyor ise;

INSRang=math.atan2(cezeri.imu.Vy,cezeri.imu.Vx) yon=INSRang	Daha güvenilir olan ataletsel yatay hız verileri ile aracın yönünü yeniden hesapla. yon'ü INSRang'e eşitle.

Açıklama: Algoritmada IMU kullanılarak Vx ve Vy kullanılarak bulunan enlem boylam değerleri neredeyse hatasız sonuç verirken, açısal hız için yüksek doğrulukta bir sonuç elde edilememiştir. Kodun mevcut güncelliğinde, simülasyon süresi uzadıkça oluşan hatanın büyüdüğü gözlemlenmiştir. O yüzden aracın düz uçuşta sadece ileriye gidiyor olmasından yola çıkarak yatay hızlar ile hesaplanan ikinci yöntem ile bu hatanın azaltılması uygun görülmüştür.

11. Senaryo 16'da şarj durumunu gözeterek şarj istasyonlarının tespiti ve hava aracının oraya yönlendirilmesini sağlayan algoritma

I. Batarya durumları veri yapısı

Bataryanın şarj durumu cezeri.batarya.veri okunarak bulunur. Bu değişken yüzde cinsinden şarj miktarını verir. Batarya'da herhangi bir arıza olup olmadığı ise cezeri.batarya.hata olarak okunur. Bu, değişken şarj edilip edilmemesi için hiç kullanılmamış olsa da, batarya arızası bir acil durum olarak değerlendirilmiş ve ana hedeften vazgeçip inilebilir en yakın noktaya iniş yapmasını tetikleyen bir koşul olarak kullanılmıştır.

II. Hangi gerilim eşiğinde şarj olunması gerektiğinin karar mekanizması

Algoritma yarışmadaki bütün senaryolar gözetilerek yazıldığından senaryo bazında bir şarj eşiği belirlenmemiştir. Şarj olmayı tetikleyen 3 kurulum değişkeni, 3 adet koşul/eşik vardır. Şarj algoritmasının kodu EK-2'de incelenebilir. Çalışma prensibi şu ana başlıklar altında incelenebilir:

1- Enablecharge=True, şarj olma algoritmasını devreye sokar. Sarjfirsatidegerlendir=True, eğer senaryo gereği indiği lokasyon bir şarj istasyonuysa bunu fırsat olarak değerlendirip şarj olur. Hizonemli=True sürenin önemli olduğu senaryolarda kullanılan değişkendir.

2- getoptimalchargestation(x,y) adlı fonksiyon, aracın mevcut konumundan ulaşmak istediği ana hedefe mesafeyi en az uzatacak şarj istasyonunu bulur. Eğer şarj %80'in altındaysa, ve şarj olmak yolu %20'den az uzatıyorsa, araç bu optimal şarj istasyonuna şarj olmaya yönelir. Ek-1'deki Senaryo 18'e ait videoda bu koşulun uygulanması görülebilir.

3- %70 altında şarj eşiğinde, %30 rezerve bırakacak şekilde gitmesi gerektiği noktaya mesafe, batarya şarjı miktarının 14 katı ise, araç hedefi ile mevcut konumu arasındaki optimal şarj istasyonuna şarj olmaya yönelir.

4- %40 altında şarj eşiğinde %30 rezerve bırakacak şekilde gitmesi gerektiği noktaya mesafe, batarya şarj miktarının 10 katı ise, araç getchargestation() fonksiyonunu çağırarak var olan en yakın şarj istasyonuna iner. Bu koşul hız önemli ise devre dışı bırakılmıştır.

5- Eğer Senaryo'da hız önemli değil ise araç %95 şarja kadar bekler, çünkü kalkış yapana kadar aracın %6-8 ekstra şarj olduğu gözlemlenmiştir. hızonemli=False iken eşikler görevleri rahatlıkla yapabilecek şekilde belirlenmiştir. hizonemli=True ise calculaterequiredbattery(array,int) fonksiyonu çağırılır. Bu fonksiyon görevde geriye kalan uçuş mesafesi ve toplam iniş kalkış sayısını hesaplayarak aracın %21-24 arası bir şarj ile görevini tamamlayacak kadar şarj olmasını sağlar. Aracın iniş kalkış için ~%4, 16m ilerlemek için %1 şarj gerektiği ilerlediği gözlemlenmiştir. Bu gelişmiş özellik sürenin önemli olmadığı senaryolarda da denenmiş, ancak önceki eşikler yüksek güvenlik payları bırakılarak şarj gereksinimi olan bütün görevleri başarıyla tamamlayacak şekilde seçilmiş olduğundan ötürü, bu fonksiyonun kullanımının senaryonun başarılı olmasına herhangi bir getirisi olmayacaktır. O yüzden sürenin önemli olmadığı durumlarda bu fonksiyon çağırılmamaktadır. EK-1'deki Senaryo-20'ye ait videoda bu şarj algoritması çalışırken gösterilmektedir.

III. Şarj İstasyonlarının veri yapısı

Şarj istasyonlarının veri yapısı `cezeri.harita.sarj_istasyonlari`[] şeklindedir. Örneğin 2. Şarj istasyonunun enlemi, `cezeri.harita.sarj_istasyonlari`[2].enlem şeklinde okunabilir.

12. Kaynakça

EUSPA. (2021, 12 3). https://www.euspa.europa.eu/european-space/eu-space-programme/what-gnss adresinden alındı

GÜNHAN, Y. (2014, 06 27). *ATALETSEL ÖLÇÜM BİRİMİ HATALARININ BELİRLENMESİ VE DÜZELTİLMESİ.* openaccess.hacettepe.edu.tr: http://openaccess.hacettepe.edu.tr:8080/xmlui/bitstream/handle/11655/2717/4994fa44-46e4-4fd8-8fd4-31403ff62d09.pdf?sequence=1&isAllowed=y adresinden alındı

Mathew, S. (2017, 04 24). *SCIENCING.* https://sciencing.com/different-parts-of-a-mercury-thermometer-12073649.html adresinden alındı

pythontr.com. (2014, 06 24). https://www.pythontr.com/makale/python-nedir-235 adresinden alındı

ReadPhysics. (2021, 05 11). Barometer: Types, Work And Definition With Photos: https://readphysics.com/barometer-types-and-work/ adresinden alındı

EK-1 Senaryolara Ait Videolar:

Yarışma şartnamesinde belirtildiği üzere, gerçekleştirilen senaryoların videolarının linkleri aşağıdadır:

Playlist: https://youtube.com/playlist?list=PLw0Ju259diKVRcKjPE9vDApr7hkxC3goP

Senaryo-01: https://youtu.be/TEkJXRWtB_4

Senaryo-02: https://youtu.be/4oRLQH9r8zE

Senaryo-03: https://youtu.be/1ypa95Ye0nM

Senaryo-04: https://youtu.be/0eLsX8Kdgy0

Senaryo-05: https://youtu.be/xsSTsxP-XWs

Senaryo-06: https://youtu.be/89rHEyOgWuQ ve https://youtu.be/z5SMYQCPkTw

Senaryo-07: https://youtu.be/Qj8984jbaEQ ve https://youtu.be/VzhtZO-TvSU

Senaryo-08: https://youtu.be/mS8AuLvHwLg

Senaryo-09: https://youtu.be/fPvUzoy5z3M

Senaryo-10: https://youtu.be/k3T0px3j8RM

Senaryo-11: https://youtu.be/PhFWHKQdlro

Senaryo-12: https://youtu.be/_h8bEdIm4oE

Senaryo-13: https://youtu.be/_N7T-TNKM0A

Senaryo-14: https://youtu.be/KwHjaZgVxM8

Senaryo-15: https://youtu.be/istT19Kze50

Senaryo-16: https://youtu.be/bSP8QxEK91Q

Senaryo-17: https://youtu.be/buBOSCmPri8

Senaryo-18: https://youtu.be/5cAduW6HIE8

Senaryo-19: https://youtu.be/0W_14D8I5-8

Senaryo-20: https://youtu.be/C_q5QcTAVNg

EK-2 Kod:

Bu raporun teslimi itibariye yarışma kodunun en güncel hali aşağıdaki gibidir:

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-
# comment koyabilmek için yukarıda ascii den utf8e geçtik koda bir etkisi yok türkçe karakter problem yaratıyordu
from cezeri_lib.cezeri import Robot, Cezeri
import rospy
import math
import time
import copy
#v3.7.1 -ek commentler eklendi, eski commentler düzenlendi
cezeri = Cezeri("quadrotor")
cezeri.senaryo = 20
followrouteback=False       # Senaryo 3 gibi aynı rotadan kalkış noktasına dönmesini istiyorsak true
preflight=True              #uçuş öncesi yol planlaması yapar varış sırası önemli olmayan senaryolarda en kısa yolu seçer
sarjfirsatidegerlendir=True #eğer indiği yer şarj istasyonuysa hemen kalkmaz şarj olur örn. 17. senaryoda faydalı
hizonemli=True              #True ise süreye veya pile dayalı senaryolarda güzel gözükmesi için yapılan hedefe yaklaşırken kademeli yavaşlama ve yumuşak iniş algoritmalarını devre dışı bırakarak uçuşu hızlandırmayı amaçlar
enablecharge=True           #senaryo şarj gerektirmiyorsa işleri hızlandırmak adına şarjı devre dışı bırakabiliriz

##########################setup kısmı ve global değişkenler
cezeri.dur()            #başlangıçta durduğundan emin olmak için
headingset=0            #yönün doğru olup olmadığını anlatan değişken. 0 ise değil, 1 ise az, 2 ise tam doğru, 2 kodda kullanılmadı
inis=False              #aracın iniş değişkeni true ise iniş yapıyor demektir.
atlocation=False        #True ise araç iniş bölgesine tam merkezlenmiştir
debug=2                 #debug değişkeni, kod yazımı sırasında farklı sayılar ile farklı özelliklerin ekrana basılması sağlandı
hedefindexi=0           #cezeri.hedefler dizisindeki hedeflerin index değişkeni
yenihedef=True          #navigasyon bölümüne yeni hedef alınabileceğini belirtir
cezeri.bekle(1)         #simülasyon bazen yüklenirken anında verileri yükleyemiyor
index=0                 #genel döngü indexi. Kod içerisinde farklı görevlerde kullanıldı.
inmekistiyoruz=True     #hedefte aracın inmek isteyip istemediğini belirten değişken
zoneavoid=False         #an itibariyle asıl hedefe yönelmek yerine engelden kaçınma manevrası yaptığını belirten değişken
charge=False            #sonraki hedefte şarj olmak isteyip istemediğimizi belirten değişken
```

```
kalkis=True
#kalkış sırasında trafiğin geçmesini beklemek için kullanılan değişken, ancak
simülasyon kalkmasa bile başlangıçta kalkmış varsayıyor ve hemen kalkış noktasında
trafik varsa başarısız sayıyor.
waypoints=0             #kaçınma manevrası sırasındaki planlanmış dö-
nüş sayısı. Haritada aracın sıkışacağı bölgeler olmadığı için kodun gün-
cel hali 1'den fazla planlama yapmamakta.
tempspeed=cezeri.yavas              #geçici hız bölge-
lerinde kullanılmak için değişken.
target=cezeri.hedefler[0]           #target değişkeni rota izlenirkenki güncel ak-
tif hedef
primarytarget=cezeri.hedefler[0]    #ana hedef değişkeni. kalkış noktasına dö-
nüş hariç target değişkenine eşdeğer.
routecomplete=False                 #ro-
tanın tamamlanıp tamamlanmadığını gösteren değişken.
prevnofly=0.0           #önceki döngüden kalan uçuşa yasak bölge uzaklığı
Closestobject=99999     #en yakın obje değişkeni
Closestangle=0          #en yakın objenin araca göre açısı
chargingnow=False       #an itibariyle şarj olup olmadığını gösteren değişken
chargeamount=100
gotoorigin=False        #rota bulma esnasında hedeflere değil, kalkış nok-
tasına gidildiğini gösteren değişken.
Mesafe=0                #genel mesafe değişkeni, birden fazla amaçla kullanıldı.
checkhzlimit=False      #aracın bulunduğu noktada hız limiti olup olma-
dığını gösteren değişken.
chiz=0                  #cezeri hız değişkeni, nor-
malde cezeri.hızlı'ya (3.0) eşitken hedefe yaklaştıkça yavaşlamak için kullanılır
mu=0                    #nofly hesapları için kullanılan bir çarpım so-
nucu değişkeni.
preflightin-
dex=0        #uçuş öncesi rota hesaplandığında bu rota takip edilirken kullanılan
index.

class NAV: #navigasyon için aracın gittiği hedefin enlem/boylam koordi-
natlarını tutan değişken.
    enlem=0
    boylam=0
nav=NAV() #aracın düz uçuşta yöneldiği koordinatlar
target=NAV() #aracın yöneldiği hedef
chargetarget=NAV() #aracın yönelebileceği şarj istasyonu koordinatları

#cezeri.imu verileri kullanılarak enlem, boylam, irtifa ve yön hesapla-
mak için gnss'den başlangıç verileri gerekli.
INSenlem=cezeri.gnss.enlem      #INS, sistem saati ve cezeri.imu verileri ile en-
lem/boylam hesabı yapar
INSboylam=cezeri.gnss.boylam
INSRenlem=cezeri.gnss.enlem     #INSR,simülasyon saati ve cezeri.imu ver-
ileri ile enlem boylam hesabı yapar
INSRboylam=cezeri.gnss.boylam
```

```
INSRirt=cezeri.gnss.irtifa
INSRang=cezeri.manyetometre.veri
startEnlem=cezeri.gnss.enlem        #simülasyon başlanıç enlemi
startBoylam=cezeri.gnss.boylam      #simülasyon başlanıç boylamı
oldirt=cezeri.barometre.yukseklik   #dikey hız hesabı için döngü içi eski irtifa
değeri ve başlangıç değeri

#fonksiyonlar
def hedefyon(X,Y,tgtX,tgtY): #hedef ve araç koordinatları ile yön bulan fonksiyon
    direct=math.atan2(tgtY-Y,tgtX-X)    #arctangent2 fonksiyonu
    return(direct)
def gnd(): #yerden yükseklik bulan fonksiyon
    if cezeri.radar.hata==False: #radar çalışıyorsa tercih et
       gn=cezeri.radar.mesafe
    elif cezeri.lidar.hata==False: #radar bozulduysa lidar çalışıyorsa tercih et
       gn=cezeri.lidar.mesafe
    else: #radar ve lidar bozulduysa irtifa verisi ve harita verisi ile bul
       gn=irt()-(cezeri.harita.bolge(Cenlem,Cboylam).yukselti)
    return(gn) #bulduğun yerden yükseklik değerini döndür
def irt(): #irtifa bulan fonksiyon
    if cezeri.barometre.hata==False: #barometre çalışıyorsa kullan
        ir=(cezeri.barometre.yukseklik)
    elif cezeri.gnss.hata==False and cezeri.gnss.karisim==False: #barometre bo-
zuk gps çalışıyorsa kullan
        ir=(cezeri.gnss.irtifa)
    else: #hem barometre hem gnss bozuksa
        ir=INSRirt #INS kullanarak bulunuan global irtifa değişkenini kullan
    return(ir) #bulduğun irtifa değerini döndür
def Dist(X,Y,tgtX,tgtY): #iki koordinat arası mesafe bulan fonksiyon
    dst=math.sqrt((Y-tgtY)**2+(X-tgtX)**2) #pisagor teoremi
    dst=dst*20 #20 ile çarparak metreye çevir
    return(dst)
def getVz(): #dikey hız bulma fonksiyonu
    if cezeri.imu.hata==False and cezeri.imu.gurultu==False: #imu çalışıyor ise
        return(cezeri.imu.Vz)
    else: #imu çalışmıyorsa irtifa ve simülasyon zamanından dikey hız hesapla
        return((irt()-
oldirt)/((float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000
000)-timeR))
def getchargestation(): #en yakın şarj istasyonu bulan fonksiyon
    EskiMes=999999
    j=0
    for i in range(len(cezeri.harita.sarj_istasyonlari)):
        Mesafe=Dist(Cenlem,Cboylam,cezeri.harita.sarj_istasyonlari[i].en-
lem,cezeri.harita.sarj_istasyonlari[i].boylam)
        if Mesafe<EskiMes:
            EskiMes=Mesafe
            j=i
    return(j)
```

```
def getoptimalchargestation(Target_en,Target_boy): #mevcut lo-
kasyon ve hedef ile en az yol uzatan şarj istasyonu
    EskiMes=999999
    j=0
    for i in range(len(cezeri.harita.sarj_istasyonlari)):
        Mesafe=Dist(Cenlem,Cboylam,cezeri.harita.sarj_istasyonlari[i].en-
lem,cezeri.harita.sarj_istasyonlari[i].boylam)+Dist(Target_en,Tar-
get_boy,cezeri.harita.sarj_istasyonlari[i].enlem,cezeri.harita.sarj_istasyon-
lari[i].boylam) #mevcut lokasyondan indexteki şarj istasyonuna + şarj istasyonun-
dan hedefe mesafe hesapla
        if Mesafe<EskiMes:
            EskiMes=Mesafe
            j=i
    return(j)

def getclosesthospital(): #en yakın hastaneyi bulan algoritma
    EskiMes=999999
    j=0
    for i in range(len(cezeri.harita.hastaneler)):
        Mesafe=Dist(Cenlem,Cboylam,cezeri.harita.hastaneler[i].en-
lem,cezeri.harita.hastaneler[i].boylam)
        if Mesafe<EskiMes:
            EskiMes=Mesafe
            j=i
    return(j)

def getclosestLZ(): #en yakın iniş bölgesi bulan fonksiyon, motor arızasında to-
prağa da inişe izin verildiği için ikincil alternatif olarak kullanıldı.
    EskiMes=999999
    j=0
    for i in range(len(cezeri.harita.inis_bolgeleri)):
        Mesafe=Dist(Cenlem,Cboylam,cezeri.harita.inis_bolgeleri[i].en-
lem,cezeri.harita.inis_bolgeleri[i].boylam)
        if Mesafe<EskiMes:
            EskiMes=Mesafe
            j=i
    return(j)
def noflytest(currX,currY,tgtX,tgtY,dst): #uçuşa yasak bölge ve engel test fok-
nsiyonu
    nfly=99999
    D=hedefyon(currX,currY,tgtX,tgtY)
    index=0
    while index<dst-2: #index dst-2 den küçükse
        nflyen=Cenlem+index*math.cos(D)/20
        nflyboy=Cboylam+index*math.sin(D)/20
#tgt yönünde index metre uzaklıktaki noktanın koordinatını bul
        if cezeri.harita.bolge(nflyen,nflyboy).ucusa_yasa-
kli_bolge==True and nfly>index: #uçuşa yasak bölge mi?
```

```
            nfly=index #koşulun sağlandığı ilk index engele olan en yakın
mesafedir.

            break #engel bulunduğuna göre döngü sonlanmalı
        if cezeri.harita.bolge(nflyen,nflyboy).ruzgar==True and nfly>index:
#rüzgar bölgesi mi?
            nfly=index
            break
        '''if cezeri.harita.bolge(nflyen,nflyboy).trafik==True and nfly>index:
#bölgede trafik var mı? Bu kısım kullanılmadı yön değiştirmeye gerek yok diğer
sağ/sol kaçma algoritması yeterli.
            nfly=index
            break                    '''
        if cezeri.harita.bolge(nflyen,nflyboy).trafik_bol-
gesi==True and cezeri.harita.bolge(nflyen,nflyboy).yukselti>cezeri.irtifa_ara-
ligi[1] and nfly>index: #bölge trafik bölgesi ise irtifa üst limitimiz var. Trafik
bölgesi ise ve irtifa sınırları içinde önümüzdeki engeli üstünden uçarak aşamıyor-
sak bu bölge de yön değiştirerek aşılması gereken bir bölge

            nfly=index
            break
        if dst>100:
            index=index+3
        elif dst>40:
            index=index+2
        else:
            index=index+1
    return(nfly)

def collisionavoid(D,dst,nf,rz,tr,trb): #D yönünde, dst mesafede isteğe bağlı tü-
rde engel olup olmadığını kontrol eden fonksiyon
    nfly=99999
    index=0
    while index<dst-2:
        nflyen=Cenlem+index*math.cos(D)/20
        nflyboy=Cboylam+index*math.sin(D)/20
        if nf==True and cezeri.harita.bolge(nflyen,nflyboy).ucusa_yasa-
kli_bolge==True and nfly>index: #nofly bulmak istiyorsak nf=True gönder

            nfly=index
            break
        if rz==True and cezeri.harita.bolge(nflyen,nfly-
boy).ruzgar==True and nfly>index: #ruzgar bulmak istiyorsak onksiyona rz=True
gönder

            nfly=index
            break
```

```
        if tr==True and cezeri.harita.bolge(nflyen,nfly-
boy).trafik==True and nfly>index: #trafik bulmak istiyorsak onksiyona tr=True
gönder
            nfly=index
            break
        if trb==True and cezeri.harita.bolge(nflyen,nflyboy).trafik_bol-
gesi==True and cezeri.harita.bolge(nflyen,nflyboy).yukselti>cezeri.irtifa_ara-
ligi[1] and nfly>index: #trafik bölgesi üst irtifa sınırından ötürü aşılamayacak
bir engel var mı kontrol etmek istiyorsak fonksiyona trb=True gönder.
            nfly=index
            break
        index=index+3 #3 metre aralıklarla kontrol et, 3-6-9-12...
    return(nfly)
def permutatepaths(arry):   #alternatif yolların denenmesi için permüta-
syon fonksiyonu arry girdisindeki bütün değerlerin farklı sıralı permütasyonu-
larını döndürür.
    if len(arry)==0:        #array girdisi boşsa boş döndür
        return[]
    elif len(arry)==1:      #array girdisi tek bir element ise kendisini dö-
ndür, bu fonksiyonun sonlanmasını sağlayacak
        return[arry]
    l=[]                    #yeni array oluştur
    for i in range(len(arry)):  #1. döngü ile ilk değeri al bul
        j= arry[i]
        rAry=arry[:i]+arry[i+1:]    #bulduğun permütasyonu listeden çıkar rAr-
ray kalan alternatifler
        for k in permutatepaths(rAry): #aynı fonksiyonu çağırarak 2. dö-
ngü ile ile 2. değeri bul bu fonksiyon içinde liste 1 element kalana kadar ken-
dini çağıracak
            l.append([j]+k)
    return l
def totalflightpath(arry): #arry ile gönderilen hedef varış sırası izlendiğinde
oluşak toplam uçuş mesafesini hesaplar.
    i=0 #sayaç
    t=0 #toplam mesafenin eklendiği değişken.
    while i<len(arry):
        if i==0: #ilk değer için istisna, başlangıç noktasından mesafe bul.
            t=Dist(startEnlem,startBoylam,cezeri.hedefler[arry[i]].en-
lem,cezeri.hedefler[arry[i]].boylam)
        else: #sonraki değerler için array dizisindeki “i ile i-1” arası mesafe
bul
            t=t+Dist(cezeri.hedefler[arry[i-1]].enlem,cezeri.hedefler[arry[i-
1]].boylam,cezeri.hedefler[arry[i]].enlem,cezeri.hedefler[arry[i]].boylam)
        i=i+1
    return t
def calculaterequiredbattery(arry, u): #arry ile gönderilen hedef varış sırasında,
biz baştan u adet hedef ilerledikten sonra geriye kalan kalkış iniş sayısı ve
mesafeyi hesaplayarak ne kadar şarja ihtiyaç olduğumuzu hesaplar
```

```
    landings=1 #iniş sayacı, son hedef olsa bile şarj istasyonundan kalkıp inmemiz
gerekeceği için 1den başlıyor.
    if u==len(arry): #bu if u son hedef ise out of bounds olmasına engel oluyor.
Zaten son hedefe mesafe hesaplanacak, ve sorgu içinde u+1 yapıldıktan sonra son-
raki while'a girmeyecek.
        u=u-1
    bdist=Dist(Cenlem,Cboylam,cezeri.hedefler[arry[u]].en-
lem,cezeri.hedefler[arry[u]].boylam)
    u=u+1
    while u<len(arry): #u array boyundan küçük olduğu sürece
        bdist=bdist+Dist(cezeri.hedefler[arry[u-1]].enlem,cezeri.hedefler[arry[u-
1]].boylam,cezeri.hedefler[arry[u]].enlem,cezeri.hedefler[arry[u]].boylam)
        u=u+1 #u arrtır
        landings=landings+1 #her ana hedef bir iniş+kalkış demek.
    capacity=landings*4+bdist/16+15
    if debug>=2:
        print("landings", landings, "bdist", bdist, "capacity", capacity)
    return capacity

if hizonemli: #hız önemliyse yakıt konusunda farklı kısıtlar konulabilir
    bingofuel1=70
    bingofuel2=40
    reserve=20 #hız önemliyse zaten gereği kadar yakıt alıyoruz, rezerve göre
yakıt alma ihtiyacımız yok.
else:
    bingofuel1=70
    bingofuel2=40
    reserve=30

timeR=0.00 #rospy simülasyon zamanı hesaplamak için başlangıç zamanı
preTime=0.00 #sistem işlemci zamanından enlem boylam hesaplamak için başlangıç za-
manı

startIrtifa=irt() #başlangıç irtifası

yenihedef=False
atlocation=False
inis=False
inmekistiyoruz=True
stops=[] #bu durulması gereken hedeflerin sıralı dizisi.
i=0
while i<len(cezeri.hedefler): #stops dizisini oluştruyor, örneğin 3 adet var ise
0,1,2 elemanlarına sahip dizi oluşturuyor.
    stops.append(i)
    i=i+1
if preflight==True: #uçuş planlaması istiyorsak en kısa yoldan ana hedeflere
gitmek için optimum yol hesaplıyor.
    alternatepaths=[] #alternatif yollar dizisi
    alternatepathlengths=[]
```

```
    alternatepaths=permutatepaths(stops) #stops dizisindeki yolların farklı diz-
ilimleri ile permütasyonu
    i=0
    while i<len(alternatepaths): #toplam uçuş mesafesi fonksiyonu ile alternate-
paths in indeksindeki sıra ile görev tamamlanır ise aynı indekste alternatepath-
lenghs'e bu yolun yolun uzunluğunu yazıyor.
        alternatepathlengths.append(totalflightpath(alternatepaths[i]))
        i=i+1
    i=0
    use=0 #hangi index en kısaysa onu kullanmalıyız
    mindist=alternatepathlengths[0] #ilk index en kısa varsayalım
    while i<len(alternatepathlengths): #tüm permütasyonlar bitene kadar
        if alternatepathlengths[i]<mindist: #eğer bu indexteki yol bir öncekinden
daha kısa ise bunu kullanmalıyız.
            mindist=alternatepathlengths[i]
            use=i #alternatepathlenghts'deki en kısa yolun indexi bu çıkacak
        i=i+1
    hedefindexi=alternatepaths[use][preflightindex] #alternatepathlengthteki en
kısa yolun indeksi, alternatepaths'deki en kısa yolun verildiği dizi, preflightin-
dex şu an 0, o da bu dizideki ilk eleman.

primarytarget=cezeri.hedefler[hedefindexi] #şimdiden atayabiliriz ana hedefi.
while cezeri.aktif():

    Vz=getVz()

    #sistem işlemci zamanından poziyon bulma
    '''INSenlem=cezeri.imu.Vx*(time.time()-preTime)/22+INSenlem
    INSboylam=cezeri.imu.Vy*(time.time()-preTime)/22+INSboylam
    preTime=(time.time()) '''

    #simülasyon zamanından enlem boylam poziyon bulma
    INSRen-
lem=cezeri.imu.Vx*((float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsec
s)/1000000000)-timeR)/22+INSRenlem
    INSRboylam=cezeri.imu.Vy*((float(rospy.get_rostime().secs)+float(rospy.get_ros
time().nsecs)/1000000000)-timeR)/22+INSRboylam
    INSRirt=cezeri.imu.Vz*((float(rospy.get_rostime().secs)+float(rospy.get_rostim
e().nsecs)/1000000000)-timeR)/1.1+INSRirt
    INSRang=INSRang+cezeri.imu.acisal_hiz*((float(rospy.get_rostime().secs)+float(
rospy.get_rostime().nsecs)/1000000000)-timeR)*1.01
    timeR=float(rospy.get_rostime().secs)+float(rospy.get_rostime().nsecs)/1000000
000
    Vz=getVz()
    oldirt=irt()
```

```
    if cezeri.gnss.hata==False and cezeri.gnss.ka-
risim==False and abs(cezeri.gnss.enlem-INSRenlem)<0.2: #eğer GNSS sorunsuz ise ve
verisi IMU ile bulunan veri ile tutarlı ise:
        Cenlem=cezeri.gnss.enlem #aracın enlemi GNSS verisi kullan
        INSRenlem=cezeri.gnss.enlem #IMU verisini de GNSS verisi ile güncelle
    else:
        Cenlem=INSRenlem #GNSS kullanılmıyor ise IMU verisini kullan

    if cezeri.gnss.hata==False and cezeri.gnss.ka-
risim==False and abs(cezeri.gnss.enlem-INSRboylam)<0.2:
        Cboylam=cezeri.gnss.boylam
        INSRboylam=cezeri.gnss.boylam
    else:
        Cboylam=INSRboylam
    if cezeri.manyetometre.hata==False and cezeri.manyetometre.gurultu==False:
        yon=cezeri.manyetometre.veri
        #INSRang=cezeri.manyetometre.veri #Buna gerek olmadı

    else:
        if (abs(cezeri.imu.Vx)>4 or abs(cezeri.imu.Vy)>4) and inis==False and Clos
estobject>60: #özetle araç ilerliyor ve sadece ileri doğru ilerliyor ise;
            INSRang=math.atan2(cezeri.imu.Vy,cezeri.imu.Vx)
        yon=INSRang #açısal hız yerine yatay hızları kullanarak yönü güncelle.

    distance=Dist(Cenlem,Cboylam,nav.enlem,nav.boylam) #mevcut yöneldiğimiz
koordinata mesafe
    Tdistance=Dist(Cenlem,Cboylam,target.enlem,target.boylam) #gitmek istediğimiz
hedefe (ana hedef veya şarj istasyonu) mesafe.
    PTdistance=Dist(Cenlem,Cboylam,primarytarget.enlem,primarytarget.boylam) #ana
hedefe mesafe.

    if cezeri.acil_durum==True: #acil durum var ise
        yenihedef=False #yönelme algoritmasından yeni hedef istemiyoruz
        inmekistiyoruz=True #vardığımız noktaya inmek istiyoruz.
        primarytarget=cezeri.harita.hastaneler[getclosesthospital()] #ana
hedefimzi getclosesthospital ile bulunan en yakın hastane indeksine sahip hastane.
    elif cezeri.motor.hata==True or cezeri.batarya.hata==True:
        yenihedef=False
        inmekistiyoruz=True
        if cezeri.harita.bolge(Cenlem,Cboylam).inilebilir==True: #motor arızasında
eğer bulunduğumuz nokta inilebilir ise buraya in.
            inis=True #bunu dediğimiz anda yatay uçuş devre dışı kalkıyor ve inişe
geçiyoruz
            primarytarget.enlem=Cenlem
            primarytarget.boylam=Cboylam #inişe elverişlilik ile ilgili diğer
kontrolleri geçmek için ana hedefimizi de bu koordinata eşitledik.
        else:
```

```
            primarytarget=cezeri.harita.inis_bolgeleri[getclosestLZ()] #eğer bu bölge inilebilir değil ise, en yakın iniş noktasına yönel, zaten bir sonraki döngülerde zaten yukarısı true olup iniş yapma ihtimalimiz var.
    elif chargingnow==False and enablecharge==True: #else = acil durumlar yok demek. eğer şarj olmuyorsak ve şarj olma izini var ise:
        if cezeri.batarya.veri<80: #şarj %80in altında ise dedik, burada batarya durum kontrolüne gerek yok çünkü zaten var ise bu yukarıdaki else gelmeden inişe geçmiş olmamız gerekir.
            Mesafe=99999
            EskiMes=99999
            j=0
            for i in range(len(cezeri.harita.sarj_istasyonlari)): #burada özetle optimal şarj istasyonu buluyoruz. Mevcut konumdan şarj istayonuna + şarj istasyonundan hedefe mesafenin en küçük değerini buluyoruz.
                Mesafe=Dist(Cenlem,Cboylam,cezeri.harita.sarj_istasyonlari[i].enlem,cezeri.harita.sarj_istasyonlari[i].boylam)+Dist(cezeri.hedefler[hedefindexi].enlem,cezeri.hedefler[hedefindexi].boylam,cezeri.harita.sarj_istasyonlari[i].enlem,cezeri.harita.sarj_istasyonlari[i].boylam)
                if Mesafe<EskiMes:
                    EskiMes=Mesafe
                    j=i
            if Mesafe<PTdistance*12/10: #eğer optimal şarj istasyonu yolu sadece %20 uzatıyor ise şarj ol
                chargetarget.enlem=cezeri.harita.sarj_istasyonlari[j].enlem
                chargetarget.boylam=cezeri.harita.sarj_istasyonlari[j].boylam
                yenihedef=False
                charge=True
                inmekistiyoruz=True
            elif cezeri.batarya.veri<bingofuel1 and Dist(Cenlem,Cboylam,primarytarget.enlem,primarytarget.boylam)>(cezeri.batarya.veri-reserve)*14: #değilse ama şarj 70in altında ise ve ana hedefe yaklaşık %15 farkı +rezerve ile yeteri kadar şarjımız yoksa optimal şarj istasyonunda şarj ol
                yenihedef=False
                charge=True
                inmekistiyoruz=True
                chargetarget.enlem=cezeri.harita.sarj_istasyonlari[j].enlem
                chargetarget.boylam=cezeri.harita.sarj_istasyonlari[j].boylam
            if cezeri.batarya.veri<bingofuel2 and Dist(Cenlem,Cboylam,primarytarget.enlem,primarytarget.boylam)>(cezeri.batarya.veri-reserve)*10: #önceki koşul ne derse desin, eğer şarj %40 altında ise, %60 tüketim farkı +rezerve ile yeteri kadar şarjımız yoksa en yakın şarj istasyonuna yönel.
                chargetarget.enlem=cezeri.harita.sarj_istasyonlari[getchargestation()].enlem
                chargetarget.boylam=cezeri.harita.sarj_istasyonlari[getchargestation()].boylam
                charge=True
```

```
            elif primarytarget.sarj_istasyonu==True and sarjfirsatideger-
lendir==True: #eğer gittiğimiz yer şarj istasyonu ise ve yukarıdaki çok düşük ba-
tarya durumu yok ise optimal şarjı yoksay direk git ana hedefte şarj ol.
                chargetarget.enlem=primarytarget.enlem
                chargetarget.boylam=primarytarget.boylam
                charge=True #şarj olma fırsatı
    else:  #acil durum yok, şarj olmaya gerek yok ise;
        if hedefindexi>=0 and hedefindexi<len(cezeri.hedefler): #out of bounds
olma.
            primarytarget=cezeri.hedefler[hedefindexi] #ana hedef ata.
            if gotoorigin==True: #doğru ise kalkış noktası ana hedef
                primarytarget.enlem=startEnlem
                primarytarget.boylam=startBoylam
                charge=False

    if charge!=True: #eğer şarj olmayacaksak hedef ana hedefe eşit.
        target.enlem=primarytarget.enlem
        target.boylam=primarytarget.boylam
    else: #şarj olmak istiyorsak hedef en uygun görülen şarj istasyonu.
        target.enlem=chargetarget.enlem
        target.boylam=chargetarget.boylam


    if inis==False: #iniş yapmıyorsak istediğimiz yönü belirle.
        direction=hedefyon(Cenlem,Cboylam,nav.enlem,nav.boylam)
    altitude=irt() #fonksiyonu tekrar terkar çağırmaya gerek yok bir değişkene
ata.

    #haritadan gelişmiş yol kontrolü başla
    if zoneavoid==False: #kaçınma manevrası yapmıyorsak
        nav.enlem=target.enlem #hedef koordinatlarına yönel
        nav.boylam=target.boylam
        nofly=noflytest(Cenlem,Cboylam,nav.enlem,nav.boylam,distance) #önümüzde
bir önceki döngüdeki hedefe olan yön ve mesafede uçulmaması gereken bir engel var
mı kontrol et.
        if cezeri.harita.bolge(nav.enlem,nav.boylam).inis_bolgesi==True:
            inmekistiyoruz==True #gitceğimiz yer iniş bölgesi ise inmek istiyoruz
    else: #eğer kaçınma manevrası yapıyorsak
        if charge!=True: #şarj olmayacaksak ana hedefe doğru engel kontrolü yap
            nofly=noflytest(Cenlem,Cboylam,primarytarget.enlem,primarytar-
get.boylam,Dist(Cenlem,Cboylam,primarytarget.enlem,primarytarget.boylam))
        else: #şarj istasyonuna yöneliyorsak bu hedefe doğru engel kontrolü yap
            nofly=noflytest(Cenlem,Cboylam,target.enlem,target.boylam,Dist(Cen-
lem,Cboylam,target.enlem,target.boylam))
        if nofly>=400: #engel 400metreden uzak ise yok kabul et.
            waypoints=0
            inmekistiyoruz=True
            zoneavoid=False
            nav.enlem=target.enlem
```

```
            nav.boylam=target.boylam
    if nofly<300 and zoneavoid==False and nofly<distance: #engel 300 metreden
yakınsa, kaçınma manevrası yapmıyorsak ve engel gideceğimiz yerden daha yakın ise;
        if nofly!=0: #0 ise senaryo başarısızdır zaten ama sıfıra bölme hatası ol-
masın.
            mu=PTdistance/nofly/3
        if mu<1:
            mu=1
        zoneavoid=True #kaçınma manevrası yapılacak
        angular=0 #açısal fark değişkeni
        mult=1 #açı 1 veya -1 ile çarpılarak sağa veya sola doğru kontrol edilecek
        if waypoints==0: #eğer kaçınma yapılacak koordinat yok ise
            while angular<=1.7:  #açı ~90 dereceden küçük ise
                testnewdirection=(direction*2+yon)/3+angular*mult #kısır döngüde
bir sağa bir sola gitmesine engel olmak için mevcut yönüne ağırlık vererek yeni
yön dene
                if mult==1: #sağa doğru bu açıyı denedik, sola doğru da dene
                    mult=-1
                else:
                    mult=1 #sola doğru denedik artık açıyı değiştir ve tekrar sağa
doğru dene.
                    angular=angular+0.15
                if prevnofly>nofly: #nofly bölgeden uzaklaşıyorsak
                    waypoint1enlem=Cenlem+(prevnofly+10)*math.cos(testnewdirec-
tion)/20
                    waypoint1boylam=Cboylam+(prevnofly+10)*math.sin(testnewdirec-
tion)/20
                    towaypoint1=noflytest(Cenlem,Cboylam,waypoint1enlem,way-
point1boylam,prevnofly+10) #bu koordinata doğru ilk engelin mesafesini hesapla
                else: #nofly bölgeye yaklaşıyorsak daha boş bir alan bulması için
mesafeleri arttır
                    waypoint1enlem=Cenlem+(nofly*mu+20)*math.cos(testnewdirec-
tion)/20
                    waypoint1boylam=Cboylam+(nofly*mu+20)*math.sin(testnewdirec-
tion)/20
                    towaypoint1=noflytest(Cenlem,Cboylam,waypoint1enlem,way-
point1boylam,nofly*mu+20)
                prevnofly=nofly #noflyı sonraki döngü için prevnofly olarak
kaydet.
                if towaypoint1>=99999: #bu koordinata doğru istenen mesafede bir
engel yok ise bu uygun bir noktadır buraya yönel
                    nav.enlem=waypoint1enlem
                    nav.boylam=waypoint1boylam
                    break
                waypoints=1 #bir dönüş noktası atandı. Bu kısım fonksiyon olarak
yazılıp waypoint-1, sonra bu noktadan waypoint 2,3,4 yapılarak kaçınmaya daha
başlamadan bütün rota belirlenmebilirdi, gerek görülmedi.
```

```
    #irtifa hedef kontrolü
    if kalkis==True: #ilk kalkışı yaparken trafik varsa bekle kalkma. Simülasyon bu kontrolü yapmıyor yerdeysek bile aynı bölgede trafik varsa başarısız sayıyor.
        if cezeri.harita.bolge(Cenlem,Cboylam).trafik==True:
            cezeri.bekle(1)
        else:
            kalkis==False


    safealt=0
    trafik=99999
    hzkontrol=False
    if distance>20: #bütün yola baştan sonra irtifa için bakmasına gerek yok yaklaştıkça görür zaten
        lookupdist=20
    else: lookupdist=distance
    ang=-0.5 #yaklaşık -30 derece
    while ang<=0.5: #yaklaşık +30 dereceye kadar

        index=0
        while index<lookupdist-1: #hemen ilerisi trafik bölgesi mi bak
            en=Cenlem+index*math.cos(direction+ang)/20
            boy=Cboylam+index*math.sin(direction+ang)/20
            if cezeri.harita.bolge(en,boy).trafik_bolgesi==True and trafik>index:
                trafik=index

            temp=cezeri.harita.bolge(en,boy).yukselti
            if temp>safealt: #eğer bölgenin yükseltisi güvenli kabul irtifadan daha yüksek ise yeni güvenli irtifa bölgenin yükseltisi
                safealt=temp
            if cezeri.harita.bolge(en,boy).yavas_bolge==True: #bölge yavaş bölge ise hız kontrolü uygula.

                tempspeed=cezeri.yavas
                hzkontrol=True
            index=index+1
        ang=ang+0.5 #yaklaşık -30, 0 ve +30 derece için üç kontrol yap.


    if trafik<99999 and cezeri.irtifa_araligi[0]>safealt: #trafik bölgesi varsa ve izin verilen irtifa aralığı yükseltiden gelen güvenli irtifadan daha yüksek ise yeni güvenli irtifa izin verilen irtifa aralığının alt limiti.
        safealt=cezeri.irtifa_araligi[0]
    else: #değilse yine de güvenli irtifa 120m. Çoğu yer trafik bölgesi istisnai yerlerde de alçaktan uçmanın bir getirisi yok.
        safealt=120
    if safealt<20:  #bu sorguya gerek yok, ancak yukarıdaki değeri değiştirmek istersek, 20 metrenin altında bir irtifayı hedeflemek istemiyoruz.
        targetalt=20
```

```
    else:
        targetalt=safealt

    if hizonemli==True: #hız önemli ise, seyir irtifası hedef+2, hız önemli değil
ise hedef+10, tırmanma süresinden kısmak amaçlanıyor.
        cruisealt=targetalt+2
    else:
        cruisealt=targetalt+10

    if cruisealt>155: #seyir irtifasının çok yüksek olmasının anlamı yok.
        cruisealt=155
        safealt=150


    #yönlenme kısmı
    cezeri.don(direction-yon) #istenen yön ile mevcut yön arasındaki farka yönel.
    if (math.degrees(yon)<-90 and math.degrees(direction)>90) or (math.de-
grees(yon)>90 and math.degrees(direction)<-90):
        cezeri.don(math.radians(180)+yon-direction) #bu iki bölgede ters yöne dö-
nmek daha etkili.

    if abs(yon-direction)<0.005 or (abs(math.degrees(direc-
tion))>175 and abs(math.degrees(yon))>175): #yön kesin doğru ise. #aşağıda +/-
180 civarında tur atmasını istemiyoruz
        headingset=2 #bunu kullanmadık.
        cezeri.don(0.0)
    if abs(yon-direction)<0.08: #yön takriben doğru ise
        headingset=1
    else:
        headingset=0   #yön doğru değil

    #irtifa ve ilerleme kısmı başlıyor
    if inis==False: #iniş yapmıyor isek.
        if altitude<targetalt: #irtifamız hedeflenenden düşük ise
            atsafealt=False #güvenli irtifada değiliz
            cezeri.yukari_git(cezeri.hizli) #hızlı yukarı git
        else: #irtifamız hedeflenen irtifadan yükek ise
            atsafealt=True #güvenli irtifadayız
            dh=abs(altitude-cruisealt) #seyir irtifası ile yükselti farkı
            if dh>cezeri.yavas: #yükselti farkı cezeri.yavastan daha hızlı ol-
masın.
                dh=cezeri.yavas
            if altitude<cruisealt: #seyir irtifasından aşağıdaysa yukarı git
                cezeri.yukari_git(dh)
            else: #seyir irtifasından yukarıdaysa aşağı git.
                cezeri.asagi_git(dh)

        if headingset>0 and atsafealt==True and Closestobject>9: #yönümüz doğru ve
güvenli irtifada isek ve çok yakınımızda bir obje yok ise.
```

```
            if hzkontrol==False and checkhzlimit==False: #iki hız kontrol koşu-
lumuz da hız sınırı bölgesi yok diyor ise
                if hizonemli==False: #hız önemli değil ise
                    if distance>20: #hedefe mesafe 20metreden fazla ise hızlı git
                        dspd=cezeri.hizli
                    else:
                        dspd=distance/15 #yaklaştıkça yavaşla
                        if dspd>cezeri.hizli: #cezeri.hızlı'dan hızlı gitme
                            dspd=cezeri.hizli
                        elif dspd<cezeri.yavas: #cezeri.yavas'tan yavaş gitme
                            dspd=cezeri.yavas
                    cezeri.ileri_git(dspd) #ileri git.
                elif hizonemli==True and distance>15: #hız önemliyse 15m mesafeye
kadar hızlı git sonra zaten iniş algoritması devir alacak.
                        cezeri.ileri_git(cezeri.hizli)
            else:
                cezeri.ileri_git(cezeri.yavas) #hız kontrol bölgesi var ise ya-
vaşla
        if distance<10 and inmekistiyoruz==True: #hedefe 10 metreden fazla yak-
laştıkysak ve inmek istiyorsak iniş algoritmasını devreye sok
            inis=True
            cezeri.dur()

    if inis==True and inmekistiyoruz==True and ((cezeri.harita.bolge(Cen-
lem,Cboylam).inis_bolgesi==True and distance<20) or cezeri.mo-
tor.hata==True or cezeri.batarya.hata==True): #inişe hazırlan
        direction=0 #yönünü sıfıra çevir hem güzel gözükür hem konumlandırması ko-
lay olur
        if distance<2 or (distance<8 and hizonemli==True): #hız önemli değilse in-
ilecek koordinatı tam ortala yoksa 8metre mesafe bile yeterli.
            atlocation=True
        else:
            atlocation=False
        if headingset==1: #yönümüzü 0a çevirdiysek
            correctionspeed=abs(Cenlem-nav.enlem)*2 #ileri/geri düzeltme hızımız
enlem farkı ile orantılı.
            if Cenlem>nav.enlem: #ileri/geri düzeltme yap
                cezeri.geri_git(correctionspeed)
            else:
                cezeri.ileri_git(correctionspeed)

            correctionspeed=abs(Cboylam-nav.boylam)*2 #sağ/sol düzeltme hızımız
boylam farkı ile orantılı.

            if Cboylam<nav.boylam: #sağ/sol düzeltme yap
                cezeri.sola_git(correctionspeed)
            else:
                cezeri.saga_git(correctionspeed)
```

```
        if gnd()>15 or (gnd()>10 and hizonemli==True): #yerden uzaksak hızlı yak-
laş
            cezeri.asagi_git(cezeri.hizli)
        elif gnd()>6 or (gnd()>3 and hizonemli==True): #yere yakınsa yavaş yaklaş
            if atlocation==True: #konumlandıysak aşağı git.
                cezeri.asagi_git(cezeri.yavas)
            else:
                if Vz<-0: #at location değil alçalmayı bırakıp iniş yerine yer-
leştir
                    cezeri.yukari_git(0.1)
                else:
                    cezeri.asagi_git(0.3)
        elif headingset==1 and atlocation==True and (gnd()<4 or (gnd()<3 and hi-
zonlemi==True)):  #bütün iniş koşulları, yön, yer ve irtifa sağlandı
            if Vz<-3: #yumuşak iniş
                cezeri.yukari_git(0.1)
            else:
                if hizonemli==False:
                    cezeri.asagi_git(0.3) #yaklaşık 1.2m/s iniş hızı
                else:
                    cezeri.asagi_git(cezeri.yavas) #yukarıdaki Vz<-3 kontrolüyle
birleşince <3m/s iniş hızı

            if Vz<-10:  #öngörülmeyen bir durumda yere çok yakınsak ve çok hızlı
yaklaşıyorsak çakılma hızlı yüksel
                cezeri.yukari_git(cezeri.yavas)
        if gnd()<0.8 and atlocation==True: #flareout
            cezeri.dur() #iniş noktasına konumlanmayı artık bırak
            if hizonemli==False:
                cezeri.asagi_git(0.3) #yaklaşık 1.2m/s iniş hızı
                cezeri.bekle(0.2)
            else:
                cezeri.asagi_git(cezeri.yavas*0.9) #Cezeri yavaşa çok yakın bir
hızla dokun yaklaşık 4m/s'ye kadar hız mümkün
            if charge==True and cezeri.harita.bolge(Cenlem,Cboylam).sarj_istasy-
onu==True and cezeri.batarya.hata==False and cezeri.batarya.veri<chargeamount:
#şarj olma koşulları
                if hizonemli==True: #hız önemliyse yeteri kadar şarj et.
                    if preflight==True: #preflight yaptıysak
                        chargeamount=calculaterequiredbattery(alternate-
paths[use],preflightindex)-6
                    else: #preflight yapmıyorsak
                        chargeamount=calculaterequiredbat-
tery(cezeri.hedefler,hedefindexi)-6
                else: #acelemiz yok ise tamamen şarj et.
                    chargeamount=95
                cezeri.bekle(1) #şarj olmayı bekle.
                chargingnow=True #şu an şarj oluyoruz
            else:  #şarj bitti veya şarj olmuyor isek;
```

```
                chargingnow=False
                atlocation=False
                if abs(Cenlem-primarytarget.enlem)<0.2 and abs(Cboylam-primarytar-
get.boylam)<0.2 and inis==True:
                    yenihedef=True #yere indik, ana hedefe yakınız ve iniş modun-
dayız artık yeni hedefe yönelebiliriz
                charge=False
                inis=False #iniş modundan çıkıldığında irtifa kontrolü devreye
gireceği için araç kalkış yapacak
            if hedefindexi>=len(cezeri.hedefler) and routecomplete==False:
                routecomplete=True #rota tamamlandıysa
                if followrouteback==True and gotoorigin==False: #geri dönüş yapmak
istiyosak ancak henüz kalkış noktası hedefimiz değil ise:
                     yenihedef=True
    elif inis==True and zoneavoid==False and inmekistiyoruz==False and dis-
tance<10: #hedefe vardık ama inmek istemiyorsak direk yeni hedef al.
        cezeri.dur()
        yenihedef=True

    if yenihedef==True and zoneavoid==False and abs(Cenlem-
cezeri.hedefler[hedefindexi].enlem)<0.1 and abs(Cboylam-
cezeri.hedefler[hedefindexi].boylam)<0.1: #yeni hedef almak istiyoruz ve mevcut
hedefe vardıysak;
        if preflight==False: #normal hedefler sırasında uçuş
            if hedefindexi<len(cezeri.hedefler) and routecom-
plete==False:                    hedefindexi=hedefindexi+1
                if hedefindexi==len(cezeri.hedefler): #rota tamamlandıysa
                    hedefindexi=hedefindexi-1 #hedef indeksini bir azalt
                    routecomplete=True
            if hedefindexi>-1 and routecomplete==True and followrouteback==True:
#followrouteback true ise ve yol bir kere tamamlandıysa geri dönüşe başla
                hedefindexi=hedefindexi-1
                if hedefindexi==-1:
                    hedefindexi=1 ##out of range hatası almamak için
                    gotoorigin=True #hedef indeksi bitti, kalkış noktasına dön
            if hedefindexi>=0 and hedefindexi<len(cezeri.hedefler): #out of range
olmadan ana hedefi mevcut hedef indeksindeki cezeri.hedeflere eşitle.
                primarytarget=cezeri.hedefler[hedefindexi]
                if gotoorigin==True: #kalkış noktasına dönüyor isek
                    primarytarget.enlem=startEnlem
                    primarytarget.boylam=startBoylam
        else: #preflight hesapları ile en kısa yoldan ilerleme
            preflightindex=preflightindex+1 #alternetif sıradaki bir sonraki
hedefe yönel
            if preflightindex<len(alternatepaths[use]):
                hedefindexi=alternatepaths[use][preflightindex]
                primarytarget=cezeri.hedefler[hedefindexi]
```

```
        yenihedef=False
        atlocation=False
        inis=False
        charge=False


    if zoneavoid==True and distance<35: #kaçınma manevrası yapıyorsak ve mesafe
35in altına düştüyse kaçınma manevrasını bırak ve hedefe yönel. Bu şekilde ver-
ilmesi, önemli bir yön değişimi olmadan sonraki döngüde tekrar nofly kontrolününü
tetikleyip yumuşak dönüşler yapmasını sağlıyor.
        waypoints=0
        inmekistiyoruz=True
        zoneavoid=False
        nav.enlem=target.enlem
        nav.boylam=target.boylam

    if inis==False and atsafealt==True: #çarpma önleme kısmı loopun en sonunda, bu
sayede diğer bütün komutların önüne geçiyor
        index=0
        checkcollision=True
        checknofly=False
        checkhzlimit=False
        while index<len(cezeri.yerel_harita): #yerel haritada 1 sektör mesafede
bir engel görülüyorsa son kontrolü devreye sok
            if cezeri.yerel_harita[index].ucusa_yasa-
kli_bolge==True or cezeri.yerel_harita[in-
dex].ruzgar==True or cezeri.yerel_harita[index].engel==True:
                checknofly=True
            index=index+1
        if cezeri.harita.bolge(Cenlem,Cboylam).yavas_bolge==True: önümüz temiz
olsa bile şu an bulunduğumuz yer hız sınırı bölgesi mi kontrol et.
            checkhzlimit=True
        if checkcollision==True:
            deg=-180
            Ctemp=99999
            Closestobject=99999
            Closestangle=0
            while deg<180: #-180'den +180'e 15derece arttırarak aracın yakınındaki
diğer araçları kontrol et
                Ctemp=collisionavoid(yon+math.radi-
ans(deg),60,False,False,True,False)
                if Ctemp<Closestobject: #en yakın objeyi bul
                    Closestobject=Ctemp #en yakın objenin mesafesi
                    Closestangle=deg #en yakın objenin araca göre açısı
                deg=deg+15

            if Closestobject>0 and Closestobject<99999: #aracın yakınında başka
bir araç var ise;
                chiz=3/(float(Closestobject)/15)
```

```
                if chiz>cezeri.hizli:
                    chiz=cezeri.hizli
                if Closestobject<30:
                    if (Closestangle>=-45 and Closestangle<=45): #ileride
                        cezeri.ileri_git(0)
                        cezeri.geri_git(chiz)
                    if (Closestangle<-135 and Closestangle>-180) or (Closestan-
gle>135 and Closestangle<180): #geride
                        cezeri.geri_git(0)
                        cezeri.ileri_git(chiz)
                if Closestobject<60 and distance>20:
                    if Closestangle<0:   #sag
                        cezeri.sola_git(chiz)
                    if Closestangle>=0:  #sol
                        cezeri.saga_git(chiz)
                else:
                    cezeri.sola_git(0)
                    cezeri.saga_git(0)
            else:
                cezeri.sola_git(0)
                cezeri.saga_git(0)


            if checknofly==True: #nofly bölgelere yandan yaklaşmalarda karşı ek
koruma önlemi.
                Csag=collisionavoid(yon+math.radians(-
90),10,True,True,False,True)
                Csol=collisionavoid(yon+math.radians(90),10,True,True,False,True)
                if Csag<Csol and Csag<10: #sağa 10 metreden yakınsak ve soldan
daha uzak isek sola git
                    cezeri.sola_git(cezeri.yavas)
                if Csol<Csag and Csag<10: #sola 10 metreden yakınsak ve sağdandaha
uzak isek sola git.
                    cezeri.saga_git(cezeri.yavas)

    cezeri.bekle(0.1)



    if debug>0: #terminale debug yazdır
        print("----------------------------------------------------------------
---------------------------------")
        print('Manyeto', float("{0:.2f}".format(math.degrees(cezeri.manyetome-
tre.veri))), "INS yon", float("{0:.2f}".format(math.de-
grees(INSRang))), 'istenen yon:', float("{0:.2f}".format(math.degrees(direc-
tion))), 'Irtifa', float("{0:.1f}".format(alti-
tude)), 'INS irt', float("{0:.1f}".format(INSRirt)), 'gnd', float("{0:.1f}".for-
mat(gnd())))
        if debug>1: #terminale navigasyon ile ilgili bilgileri yazdır
```

```
            print('NAV MESAFE', float("{0:.1f}".format(dis-
tance)), "HEDEF MESAFE", float("{0:.1f}".format(Tdis-
tance)), "ANA HEDEF MESAFE", float("{0:.1f}".format(PTdis-
tance)), "hedef adedi", len(cezeri.hedefler), "hedefindexi", hedefindexi)
            print("GNS enlem/boylam", float("{0:.2f}".format(cezeri.gnss.en-
lem)), float("{0:.2f}".format(cezeri.gnss.boylam)), "IMU En-
lem/Boylam", float("{0:.2f}".format(INSRenlem)),float("{0:.2f}".for-
mat(INSRboylam)),"ana hedef enlem/boylam", float("{0:.2f}".format(primarytar-
get.enlem)), float("{0:.2f}".format(primarytarget.boylam)), 'nav en-
lem/boylam', float("{0:.2f}".format(nav.enlem)), float("{0:.2f}".for-
mat(nav.boylam)) )
            if debug>2: #terminale algoritmanın mantıksal değişkenlerini yazdır
                print("VARIABLES: inis", inis, " atlocation", atloca-
tion, "inmekistiyoruz", inmekistiyoruz, "yenihedef", yenihe-
def, "gotoorigin", gotoorigin, "hedefindexi", hedefindexi)
                print("VARIABLES: nofly1", noflytest(Cenlem,Cboylam,nav.en-
lem,nav.boylam,distance), "nofly2", noflytest(Cenlem,Cboylam,primarytarget.en-
lem,primarytarget.boylam,Dist(Cenlem,Cboylam,target.enlem,tar-
get.boylam)), "zoneavoid", zoneavoid,"mu",float("{0:.1f}".for-
mat(mu)), "closestobject", Closestobject, "closest angle", Closestangle)
                print("VARIABLES: charge", charge, "chargingnow", charg-
ingnow, "yavas bolge", hzkontrol, checkhzlimit)
        if preflight==True:
            '''a=0
            while a<len(alternatepaths):
                print("path:", alternatepaths[a], "pathlength", alternatepath-
lengths[a])
                a=a+1}
            print("--------")'''
            print("PATHPLANNING: shortest path", alternatepaths[use], "Mini-
mum pathlength", mindist)
```

